



# **Design and Implementation of a Vision-Guided Robotic System for Automated Vegetable Sorting**

**Course Code: CSE 400B**

**Fall 2025**

## **Capstone Report**

**Submitted to:**

**Rashedul Amin Tuhin**

**Senior Lecturer**

**Department of Computer Science and Engineering**

**Submitted by:**

| <b>Name</b>                 | <b>ID</b>            |
|-----------------------------|----------------------|
| <b>Fathhur Rahaman Sams</b> | <b>2022-2-60-041</b> |
| <b>Sanjana Kazi Supti</b>   | <b>2022-2-60-036</b> |
| <b>Md. Junaeid Ali</b>      | <b>2022-2-60-127</b> |
| <b>Mahfuj Alam Imon</b>     | <b>2022-2-60-131</b> |

# Introduction

In the rapidly changing scenario of modern agriculture and food processing, vegetable sorting still remains a labor-intensive and error-prone operation, leading often to inefficiencies, increased operational costs, and substantial food losses through fluctuating quality assessment. Being perishable food commodities, vegetables must be sorted on the basis of freshness in a timely and error-free manner to maintain integrity in the supply chain, ensure consumer safety, and prevent estimated yearly economic losses in billions of dollars worldwide. The combination of artificial intelligence (AI) and robotics offers a groundbreaking solution in the form of real-time automated sorting technology which has the capacity to identify defects such as rot or spoilage with high accuracy, thereby enhancing productivity, reducing manmade errors, and upholding environmentally friendly practices within an industry faced with mounting pressure from population growth and climate uncertainty.

Recent advancements in robotic and computer vision technology have paved the way for revolutionary applications to automate crops. For instance, Zhang et al. [1] provided a CNN-based model for detecting fruit freshness through hyperspectral imaging, with impressive accuracy under lab conditions. In yet another instance, Li and Wang [2] presented deep learning models for vegetable classification for conveyor belt systems emphasizing edge computing for low-latency outputs. In robotic grasping, Kim et al. [3] have developed an AI-powered gripper system for the sorting of apples based on YOLO object detection. Patel et al. [4] have studied multi-modal sensor fusion in robot arms for tomato grading and touch feedback importance. Advances in SSL have also found application in agricultural imaging, such as by Garcia and Hernandez [5], who used contrastive learning to improve model generalization with small labeled datasets for crop disease classification. Research on robotics integration consists of that by Singh et al. [6], who incorporated reinforcement learning with vision-controlled arms for berry picking. Subsequently, Chen et al. [7] highlighted real-time defect detection of vegetables using transformer-based architectures, while Oliveira et al. [8] presented SSL approaches for feature learning without supervision in the context of fruit quality assessment. Applications targeting the robotic arm, i.e., by Yamamoto et al. [9], covered path planning for sorting action in dynamic settings. Finally, more universal systems like Rahman et al.'s [10] potato sorter hybrid AI-robot system and edge-AI by Fernandez et al. [11] for mixed fruit and veg handling demonstrate the efficiency of scale.

Despite these developments, several critical gaps exist in the literature that hinder the application of independently operated vegetable sorting machines. Existing methods are mostly reliant on supervised learning models that use extensive labeled datasets, preparation of which is time-consuming and burdensome, particularly for diverse vegetable varieties that are of varying freshness levels. Besides, the majority of the studies focus on individual aspects like detection or manipulation without end-to-end integration on an optimal level, leading to inferior real-time performance under uncontrolled settings like processing lines. To add to the above, there is a great lack of systems that apply self-supervised learning to realize multi-class scenarios covering both fresh and spoiled samples within vegetable categories, resulting in poor adaptability and sub-industry standards accuracy. Moreover, there is underutilization of integration of low-cost robotic arms with in-situ cameras for precise pick-and-place operations. There is a

tendency to overlook issues like occlusion, varying lighting, and mechanical precision in high-speed sorting.

To bridge these constraints, this work answers the following research questions:

(RQ1) : How may self-supervised learning be enabled strongly enough to enhance the detection accuracy of fresh and spoiled vegetables on varied classes with extremely minimal labeled samples?

(RQ2) : What reforms must be instituted in AI models to obtain over 95% accuracy in real-time sorting with varying conditions?

(RQ3) : How does an AI agent paired with a robotic arm, such as the xArm 1s, optimize pick-and-place effectiveness in a safe and accurate way for vegetable handling?

(RQ4) : What are the environmental variables, such as lighting variation and conveyor speed, and how do these influence the system's overall performance, and how can they be alleviated?

Here, we present an end-to-end solution to these gaps by first curating a tailored dataset of eight vegetable types with each type divided equally into fresh and rotten samples (500 samples per type), and a multi-class subset of 480 samples encompassing all classes. Self-supervised learning strategies will be employed to pre-train models on this data to get robust feature extraction, followed by fine-tuning for good classification of freshness statuses. The resulting detections will then control an AI controller for the xArm 1s robot arm, which will have a built-in camera for object localization at runtime to enable autonomous picking and sorting into desired target boxes with a target accuracy of more than 95%.

The rest of the paper is organized in the following manner: Section 2 presents related work in vegetable detection, freshness assessment, and robotic agriculture automation. Section 3 describes dataset collection, preprocessing pipeline, self-supervised learning method, and robot system integration. Section 4 presents experimental results in the form of accuracy metrics, and real-time testing evaluations. Section 5 handles bigger implications, limitations, and ethical considerations, and Section 6 concludes with key takeaways and future work directions.

## Objective

1. Develop a robust self-supervised learning technique tailored for multi-class vegetable freshness categorization with the aim of significantly enhancing accuracy utilizing extremely low numbers of labeled samples by leveraging unsupervised pre-training techniques over diverse collections of fresh and rotten vegetables.
2. Create and implement architectural modifications to AI models, e.g., optimizations for real-time computation and suitability with robotic agents like the xArm 1s, to achieve more than 95% sorting accuracy and enable efficient, safe pick-and-place operations under varied dynamic conditions.
3. Describe how changes in important environmental conditions such as illumination and conveyor belt speeds influence overall system performance, and recommend pragmatic measures to prevent such occurrences in order to increase reliability and scalability in real-world use of agricultural sorters.

## Related Works

In 2024, Khanna et al.[1] introduced a new model named Fruit and Vegetable Detection Network (FVDNet) , an ensemble version of YOLOv8n for detection. For agricultural modernization they focused on the automation of fruit and vegetable detection, and also created a new dataset called FRUVEG67 that contains images of 67 different fruit and vegetable classes collected locally. Since only a few images were annotated manually and others were done by semi-supervised data annotation algorithm (SSDA) to automatically generate bounding boxes for the remaining samples, which reduced the labeling cost and time. On this dataset they applied the model (FVDNet) of three different grid configurations, an averaging strategy for bounding box prediction, and a voting system for class prediction and achieved a mean average precision (mAP) of 0.82 across all 67 classes, showing reliable detection and localization performance. For improving the accuracy on small objects, they combined Jensen–Shannon Divergence with focal loss as the main loss function. Overall, this study contributed a new dataset, a novel annotation approach, and an improved detection architecture for fruit and vegetable recognition in practical scenarios.

Li and Shi (In 2024) proposed [2] a semi-supervised object detection method designed for Brassica Chinensis growth monitoring. The model is based on a Detection Transformer (DETR) framework. DETR applies a bipartite matching strategy for label assignment and no need for traditional Non-Maximum Suppression. This choice solves the issues of occlusion, complex backgrounds, and label noise that are common in Brassica Chinensis images. The dataset was self-constructed and included two categories plant growth monitoring and wormhole detection. They also introduced two techniques : low threshold filtering and decoupled optimization for reducing the class imbalance problem and tackles conflicts between different tasks such as plant detection and wormhole detection. Here the challenge was the high labeling cost, but the semi-supervised approach allowed the use of only 5% of the annotated data while still achieving high accuracy. The method achieved an mAP of 74.1% for plant detection using just 18 labeled images and an AP50 of 73.7% for wormhole detection using 73 labeled images. Overall, this study contributed an effective semi-supervised detection pipeline that reduces annotation costs while maintaining strong performance.

Bu et al. (2025) introduced [3] a fruit and vegetable detection model, FVRT-DETR, for improving accuracy under adverse agricultural environments. The model utilized a large dataset named FV40 containing 14,511 images with 40 classes.

The model has a multi-scale deep feature fusion (MDFF) encoder and Mamba backbone, for better feature extraction and for multi-scale detection performance. Based on the Transformer architecture, this model is endowed with an end-to-end real-time detection model. The results indicate that FVRT-DETR with MDFF achieves 62.5 AP on small, 74.0 on medium, and 80.5 on large objects, which outperforms previous state-of-the-art techniques like Bi-FPN and HS-FPN. FVRT-DETR exhibited better balanced accuracy, efficiency, and robustness than YOLOv10 and other DETR variants despite occlusion and difficult to process backgrounds. Through ablation experiments, it is confirmed that the MDFF encoder contributes to detect small objects and the Mamba backbone improves the overall computation efficiency. The model has good performance on large-scale agricultural datasets where dealing with multi-scale

objects and dense configurations are extremely challenging tasks. This approach demonstrates the potential for application in real-world scenarios for automated agribusiness monitoring and provides a scalable solution for different real-world scenarios.

In 2022, García et al.[4] presented a low-cost RGB-D camera-based scheme for vineyard image semantic segmentation. The study addressed the limitation of the lack of high-quality labeled data through the usage of only 85 labeled images to train. For optimizing performance, three semi-supervised learning techniques:

PseudoLabeling, Distillation, and Model Distillation were utilized to capitalize on 320 non-annotated images. Several deep models were experimented including DeepLabV3+-ResNet50, Pan-Resnet50, HRNet and all the versions of the Unet and Unet++ architectures. The best overall model was HRNet, with a total MSA of 85.91%. On the other hand, the best overall accuracy was 84.78% with DeepLabV3+ with the ResNext50 backbone. For detection of certain grape bunches, Manet architecture with EfficientNetB3 backbone achieved 85.69% accuracy. Semi-supervised learning was added to improve segmentation performance by 5.6–6%, which validated the contribution of unlabeled data. The work further established the effect of fine-grained manual annotation on performance as well as computational time needs compared. Overall, the technique proved that accurate vineyard image segmentation can be achieved with even low-cost cameras and limited annotated datasets. The challenges addressed are low image quality, limited annotated datasets, and class imbalance, and thus the work is relevant to precision viticulture.

Bogomasov et al. (2025) proposed an iterative semi-supervised training method [5] for fruit and vegetable detection and enumeration for shopping. They claim that traditional object recognition models have to be retrained entirely when new classes or data are introduced, requiring time and large-scale manual annotation. For that, the paper uses a semi-supervised methodology in which a small subset of labeled images train the model initially and the remaining unlabeled data are progressively iteratively pseudo-labeled to enhance the model. The process reduces human labeling effort by over 60% without compromising counting accuracy at 83% on 36 heterogeneous classes of fruit and vegetables. The process aims to load costs using fewer resources and reduces annotation costs considerably. Iterative improvement also allows the model to learn quickly and from unseen data without the need for retraining. It is more responsive, faster, and high-accuracy compared to the fully-supervised conventional alternatives. Its purpose is different from semi-supervised learning's broad applicability in retail operations and agriculture robotics. Generally, the research demonstrates an efficient low-cost, accurate, and scalable object detection and counting model.

Li et al. [6] presents D<sup>3</sup>-YOLOv10, a light-weight variant of YOLOv10, in 2024 with a focus towards detecting tomatoes in greenhouses. By optimizing network architecture and utilizing multi-scale feature fusion, and utilizing DyFasterNet module to get local features, the D-LKA attention mechanism to pay attention to distorted tomato forms and occlusion regions and dynamic FM-WIoU regression loss for high boundary accuracy and convergence, the model got higher detection accuracy (an mAP<sub>0.5</sub> of 91.8%, with 54.0% reduction in parameters and 64.9% reduction in FLOPs) without losing real-time efficiency. The model used self-made tomato dataset to handle sample class imbalance issue by applying two-stage category balancing technique based on diffusion models. The model utilized knowledge distillation approach to transfer learned knowledge from a high-capacity model (YOLOv10) to a lightweight one (

D<sup>3</sup>-YOLOv10).The work highlights occlusion and close plantpacking difficulties and offers an efficient supervised solution to facility agriculture.

Johanson et al. (2024) introduced [7] a model S3AD (Semi-Supervised Small Apple Detection) to apple detection from orchard environments . The proposed algorithm addresses the most difficult problem of fruit detection, where apples are small compared to the image size and are usually occluded by leaves and stems. They developed the MAD dataset comprising 105 labeled images of 14,667 annotated instances of apples and 4,440 unlabeled images, highlighting the difficulty in harvesting large amounts of labeled data from real world orchards.Model S3AD combines contextual attention with selective tiling, enabling the system to focus better on small targets with less unnecessary computational overhead. The model S3AD achieved a mean Average Precision (AP) of 0.423 on MAD and 0.518 on MSU against which many fully supervised baselines are significantly outperformed. The novelty of the design is that its semi-supervised learning setup allows it to utilize unlabelled data and its tiling mechanism for small object detection boosting without sacrificing efficiency. Moreover, the data itself was hard due to the variations in illumination, occlusion, and limited number of annotated images, but the proposed system could attain high robustness. This paper demonstrates how semi-supervised methods can reduce annotation costs while enhancing detection accuracy in challenging agricultural environments.

Das et al[8]. introduced the Fresh CARROT dataset to classify good and bad carrot samples using 500+ images captured in natural daylight. Although no baseline model was reported by the authors, related research has applied deep learning to similar tasks. Xie et al. proposed CDDNet , achieving 99.82% accuracy for binary carrot defects and 93.01% for multi-class defects. Jahanbakhshi et al. developed an improved CNN with mix pooling, reaching 99.43% accuracy in classifying carrot shape irregularities. The novelty of these architectures lies in using lightweight CNNs, attention mechanisms , and pooling strategies that make them robust and suitable for real time grading. In particular , CDDNet was designed for fast performance in carrot defects such as cracks, discoloration, and shape variations that are visually hard to separate . The FRESH CARROT dataset reduces this issue by providing consistent backgrounds and varied carrot samples for reliable training.

Tripathi et al[9]. introduced the fruits and vegetables dataset with 20 classes collected over five months using Nikon DSLR under varying lighting and weather. The dataset description does not mention any model name tested on it . As a result, no accuracy result has been reported by the authors. There is also no architecture novelty described, since the dataset was only published for community use. The novelty instead lies in capturing diverse images with cropping and resizing to standardize input.A key dataset challenge is that many fruits and vegetables look similar in color and shape, which increases misclassification risk.

Rahaman et al[10]l. introduced the Tomato dataset with over 500 images divided into good and bad classes. The dataset description itself does not mention any specific model name or baseline classification approach applied by the authors. As a result , no accuracy result has been officially reported for this dataset. The novelty here is not in a model architecture but in how the dataset was created images were taken with a Redmi 9 power camera under daylight , against a uniform background to ensure clarity and consistency. Such a setup reduces noise and gives a solid base for testing different deep learning models. However, the dataset presents several challenges , since defects in tomatoes can be very subtle , like small

spots, color changes, or minor deformities that are hard for both humans and models to distinguish. Another challenge is that the dataset is relatively small, making it prone to overfitting if complex models are used .

Pachon Suescun et al[11]. introduced the Spoiled and Fresh Fruit Inspection dataset, containing 16,000 images of 8 different fruits, half fresh and half spoiled. The authors do not specify a model name .Consequently, there is no reported accuracy result or benchmark performance in the dataset paper. The novelty of the dataset lies in its design: images include variations in background , fruit rotation, camera distance, and lighting to make the dataset more robust and realistic. One major data set challenge is that spoilage can manifest very differently across fruit types ,making defining visible spoiled vs fresh more complex. Another challenge is that the diversity increases intra variability which complicates classification.

Muresan et al[12]. used the Fruits-360 dataset which has about 90,000+ images of fruits and vegetables in 131 classes. They applied DenseNet-201 as one of the main models for classification tasks on this dataset. In clean conditions , their best model achieved 99% accuracy on test images. When they inverted training/testing sets , DenseNet-201 still performed best, but accuracy dropped to about 96.16%. The novelty of the approach lies in using transfer learning with deep CNNs , comparing several models(NasNet-Mobile,Xception, Inception-ResNet-v2, etc), and also assessing robustness by adding Gaussian white noise to test images. Key dataset challenges include that images can be noisy or of lower quality , fruits of similar appearance causing confusion, and models being sensitive to image distortions or noise. Also , the dataset has uniform background and single fruit per image , which helps simplicity but makes generalization to real scenes harder.

Abdelrahman16 et al[13]. worked on classifying fruits and vegetables using the Fruits and Vegetables Image Recognition Dataset, which contains many classes of fruits/vegetables. The model they built is called the ResNet Pretrained Model, among others . The best result reported was 95.83% accuracy using the ResNet pretrained model. The architecture novelty isn't very deep-instead of inventing a wholly new network, they leveraged a pretrained ResNet, combined with regularization techniques, data augmentation, and comparing it with simpler classifiers. One of the important architecture innovations is exploring transfer learning to improve accuracy over simpler models, plus using data preprocessing and augmentation to help generalization. The dataset challenges include a large number of classes(131), very varied fruit and vegetable type, color/shape similarity among some classes, class imbalance and varied visual differences. Moreover, images may differ in lighting, orientation and background, which makes consistent feature extraction harder.Finally, ensuring that the model generalizes given the diversity is a core challenge in this classification task.

Rezeli et al [14]. released the YOLO Vegetable Dataset, intended for training object detection models to recognize and locate vegetables. The dataset provides annotated images for various vegetable classes, often used in detection tasks. However, the dataset page itself does not report a specific model name tested on it, nor does it give a baseline accuracy result. Because of that, architectural novelty in works using this dataset is not well documented . One consistent challenge with the dataset is variation in lighting , background clutter, and differences in vegetable appearance that make detection harder. Another issue is class imbalance- some vegetables types have more images than others, which can bias learning.

Additionally, vegetable occlusion and varied orientations add further complexity. Without published benchmark results, researchers using this dataset often struggle to measure progress or compare architecture reliably.

Ahmed et al.[15]. compiled the vegetable image dataset, which has 21,000 images spanning 15 vegetable classes, with exactly 1,400 images per class. In their comparative study , DCNN-Based vegetable image classification using transfer learning. In a comparative study they tested multiple models including VGG16 among others. The model VGG16 achieved about 99.8% accuracy and 99.7% testing accuracy. The architecture novelty lies mainly in using transfer learning from a large pretrained CNN rather than building a network from scratch, which improves performance and reduces training time. One of the dataset challenges is that many classes that sometimes look similar, distinguishing among them becomes hard. Also, while the dataset is balanced in terms of number of images per class, variability of lighting, background conditions, vegetable orientations, and small variations in vegetable appearance still pose difficulties. Another challenge is that models can overfit if data augmentation or good regularization are not used, because the dataset, though large , may not cover all environmental variations you see in real scenes.

Tseng et al. [16] contrasted SSL approaches for crop and weed identification with an emphasis on pseudo-labeling, consistency regularization, and data augmentation methods. They attained 90.5% accuracy in low-label scenarios and thus established the potency of SSL when annotation capital is limited. Novel architecture is embodied in adaptive teacher–student consistency modules, which enhance knowledge transfer in situations where supervision is limited. Key dataset challenges included domain changes between domains, changing lighting, and heterogeneous weed populations. The study highlights the trade-off between cost and performance when reducing labels, with design guidelines for scalable crop monitoring systems. The authors also suggest that applying generative augmentation would offer greater robustness in future work.

Deng and Lu [17] advanced SSL-based weed detection to an in-domain and cross-domain agriculture-optimized model. The model performed 91.3% mAP under field-to-field domain shift, outperforming fully supervised rivals in terms of generalizability. Their model blended pseudo-labeling with consistent regularization, which provided resistance to data heterogeneity. Data set problems included variability in lighting, occluding vegetation, and diverse soil-background contrasts. Experimental findings confirmed the efficacy of SSL over traditional supervised approaches to crop monitoring. Of particular note, authors emphasized SSL potential in adaptive and scalable applications, sketching active learning as a pathway to continuous model update in real-world field deployments.

Saleh et al. [18] countered difficult detection scenarios such as shadow bias and data insufficiency by proposing a hybrid SSL framework that coupled pseudo-labeling with adversarial data augmentation. The model achieved 90.8% detection accuracy with the incorporation of shadow-invariant feature extraction modules as the driving factor. Problems with the datasets included uneven lighting, shadow interference, and intra-species similarity among weeds. Their tests indicated improved generalization over different field conditions, particularly in shadow-contaminated ones. The study emphasized the need for robust learning strategies in agri-AI and argued that future work had the potential to extend the methodology toward multi-class weed detection, further enhancing SSL's ability to combat field heterogeneity.

Li et al. [19] gave comparative comparisons of a number of SSL methods for multi-class weed detection, where particular emphasis was placed on consistency regularization and pseudo-labeling. Their top-performing hybrid SSL model with teacher–student networks saw 91.9% mAP over highly diverse datasets. The study focused on difficulties from the dataset, such as class imbalance, intersecting weed types, and field condition diversity. The outcomes indicated that hybrid SSL solutions performed better than single-method solutions consistently, thereby offering flexibility in precision agriculture. Furthermore, the authors highlighted the potential in integrating SSL with UAV surveillance, ushering in a bright future towards extensive, cost-effective deployment in crop management.

Chen et al. [20] introduced preference optimization through contrastive divergence for SSL-based crop–weed classification. Through converting reward models to negative log-likelihood estimators, the model attained 89.5% accuracy for crop–weed classification using sparse labeling. The novelty of the architecture was in applying preference-based learning to steer feature adaptation, allowing models to capture expert-derived priorities. Challenges with the datasets were in the form of scarce labeled samples and diverse crop species. The research showed enhanced explainability and adaptability, with emphasis on the ability to frame model behavior to fit domain-level needs. The authors proposed the integration of this preference-guided learning into generative modeling as a future direction to further build on robustness and explainability.

Shen et al. [21] proposed WeedNet, a foundation model-based weed species classification model for global-to-local classification. By employing large-scale pretraining followed by localized feature adaptation, WeedNet achieved 92.8% accuracy under multi-scenario agricultural environments. The approach successfully handled intra-species variations and was able to perform real-time inference at 25 FPS, which is critical for UAV-based applications. Problems in the dataset were occlusion, overlapping of weeds, and illumination variations. The work focused on the scalability of the foundation model in precision agriculture and presented WeedNet as a deployable tool for autonomous weed management. Future work involves adding a few-shot adaptation to manage the newly emerging weed populations in ever-changing farm environments.

Li et al. [22] introduced a diffusion-based SSL model that incorporates generative attention mechanisms for sparse annotation-based weed detection. Their model achieved 90.7% mAP, addressing challenges such as multi-species weeds, occlusion, and varying lighting successfully. The new architecture combined generative diffusion processes with attention-based feature extraction and achieved much better robustness compared to traditional SSL approaches. Experiments demonstrated the model's potential for real-time monitoring and field-scale deployment. The study also emphasized the applicability of generative modeling to actual agricultural detection scenarios, with possible future applications to UAV images for regular, autonomous monitoring of crop fields.

Steinbrener et al.[23] developed a modular system using Mask R-CNN and LSTM networks to estimate food volume from smartphone videos. Their model achieved a volume estimation error (MAPE) of 16.0% without needing a reference object in the scene. The architectural novelty lies in fusing monocular video with the phone's inertial data (IMU) and using unsupervised learning to predict camera motion. Because no public dataset included both video and IMU data for this task, they created their own, recording 1,320 videos of 55 items across 11 fruit and vegetable classes. This dataset presented challenges like varying

backgrounds, recording speeds, and distances, and it was preprocessed into 6,909 short video segments. A key research gap they addressed is that previous methods required specialized hardware like depth cameras or a reference object for accurate volume measurement. Their work proves that a standard smartphone is sufficient, making dietary tracking more practical. The self-collected dataset was made public to support further algorithm development.

Wang et al.[24] developed a self-supervised learning framework called CC-SSL for crop classification that requires very few labeled samples. Their model, which combines the SimCLR and SCAN algorithms into a new method named Sim-SCAN, achieved a high overall accuracy (OA) of 91.44% in Ganzhou County and 81.38% in Kuitun County after data augmentation. The key architectural novelty is the design of a rich 3D input tensor that visually represents multi-temporal crop growth patterns, alongside a sample processing module that uses data augmentation to balance class distribution. The dataset, built from Sentinel-2 satellite imagery, presented challenges including severe class imbalance and the need to convert spectral time-series data into an effective visual format for the model. This work addresses the significant research gap of heavy reliance on large, expensively labeled datasets for supervised learning in agricultural remote sensing. The authors collected and used 15,186 field samples from Ganzhou and 60,491 from Kuitun, which were preprocessed into pixel-level datasets and split 9:1 for training and testing. The study demonstrates that the CC-SSL framework can achieve performance comparable to supervised models like ResNet-18 while using only a fraction of the labeled data. This approach significantly reduces the cost and resources needed for large-scale crop mapping.

Najafian et al. [25] developed a semi-self-supervised learning method for wheat head detection using the YOLO model, which achieved a high mean Average Precision (mAP) of 0.827. The key architectural novelty is a pipeline that generates a large, computationally annotated dataset by pasting augmented wheat heads from just three manually labeled images onto background scenes extracted from video clips. A major dataset challenge was the significant domain gap between these simulated images and real field conditions, which was addressed through a two-step domain adaptation process using rotated and heavily augmented versions of the original labeled images. The data was preprocessed by extracting frames from short videos of wheat fields and empty backgrounds, with only one frame per wheat video being manually annotated. This work addresses the critical research gap of heavy dependence on large, expensively annotated datasets for training effective deep learning models in agriculture. The method demonstrates that high performance is achievable with minimal manual labeling by leveraging easily acquired video data and computational annotation. The approach is not limited to wheat and can be generalized to other crops, significantly lowering the barrier for developing object detection systems in new domains. The final model's performance surpassed a baseline model trained directly on a large human-annotated dataset, highlighting the effectiveness of the proposed semi-self-supervised strategy.

Al Mamun et al. [26] conducted a systematic review of self-supervised learning (SSL) models for plant disease detection (PDD), analyzing their performance and challenges. The review did not propose a single model but synthesized various SSL approaches, which generally achieve high accuracy, such as 99.7% on the PlantVillage dataset. The architectural novelty discussed is the categorization of SSL methods into four types: generative, predictive, contrastive, and hybrid, which learn from unlabeled data. A primary dataset challenge is the scarcity of large, annotated datasets for training, with models often relying on limited public datasets like PlantVillage or custom collections. The data preprocessing methods noted include standard techniques like resizing and augmentation, but the review emphasizes the lack of diverse, real-world data. This work addresses the research gap of there being no prior systematic review dedicated solely to SSL applications in PDD. The authors conclude that while SSL is promising for reducing annotation dependency, key challenges like model generalizability, high computational

complexity, and a lack of comparative validation among SSL methods remain. Future work should focus on developing more robust and efficient SSL frameworks tailored for agricultural settings.

Gupta et al. [27] proposed Time2Agri, a self-supervised learning framework that introduces three novel temporal pretext tasks for agricultural monitoring. Their Future-Frame Prediction (FF) model achieved a high performance of 69.6% IoU for crop mapping on the SICKLE dataset, while their Temporal Frequency Prediction (FP) model reduced crop yield prediction error to 30.7% MAPE. The key architectural novelty is the design of agriculture-specific pretext tasks—Time-Difference Prediction, Temporal Frequency Prediction, and Future-Frame Prediction—that explicitly model the seasonal cycles of crops, unlike generic methods like masked autoencoding. A primary dataset challenge was the lack of large, temporally dense satellite image time series focused on agriculture, which they addressed by creating new regional (Tamil Nadu) and national (India) pretraining datasets from Sentinel-2 imagery. The data was preprocessed into monthly-resolution image chips and augmented with per-pixel dominant frequency maps derived from NDVI time series. This work addresses the significant research gap that existing remote sensing foundation models overlook the unique, predictable temporal dynamics of agricultural landscapes. A critical finding was that regional-scale pretraining outperformed national-scale pretraining, challenging the conventional foundation model paradigm and emphasizing the need for geographic specialization in agriculture. The study demonstrates that embedding domain knowledge directly into the pretext task design is more effective than applying generic computer vision methods to agricultural problems.

Wang et al. [28] proposed a novel method for fruit and vegetable classification using traditional machine learning with handcrafted features. Their model, which used a Support Vector Machine (SVM) classifier, achieved a high accuracy of 96.25%. The main architectural novelty is the combination of three types of handcrafted features: statistical texture, wavelet transform, and shape features and the use of a dimensionality reduction technique called Diffusion Maps (DM) to remove redundant information from these combined features. A key dataset challenge was ensuring the extracted features could effectively represent the different classes; the images were preprocessed with Gaussian filtering to reduce noise, converted to grayscale, and then turned into binary images to improve quality before feature extraction. This work addresses the research gap of reliance on manual sorting by providing an automated, efficient alternative that avoids the high cost and complexity of deep learning models. The study demonstrates that a carefully designed pipeline using traditional feature engineering and machine learning can achieve performance competitive with more complex deep learning approaches for classification tasks.

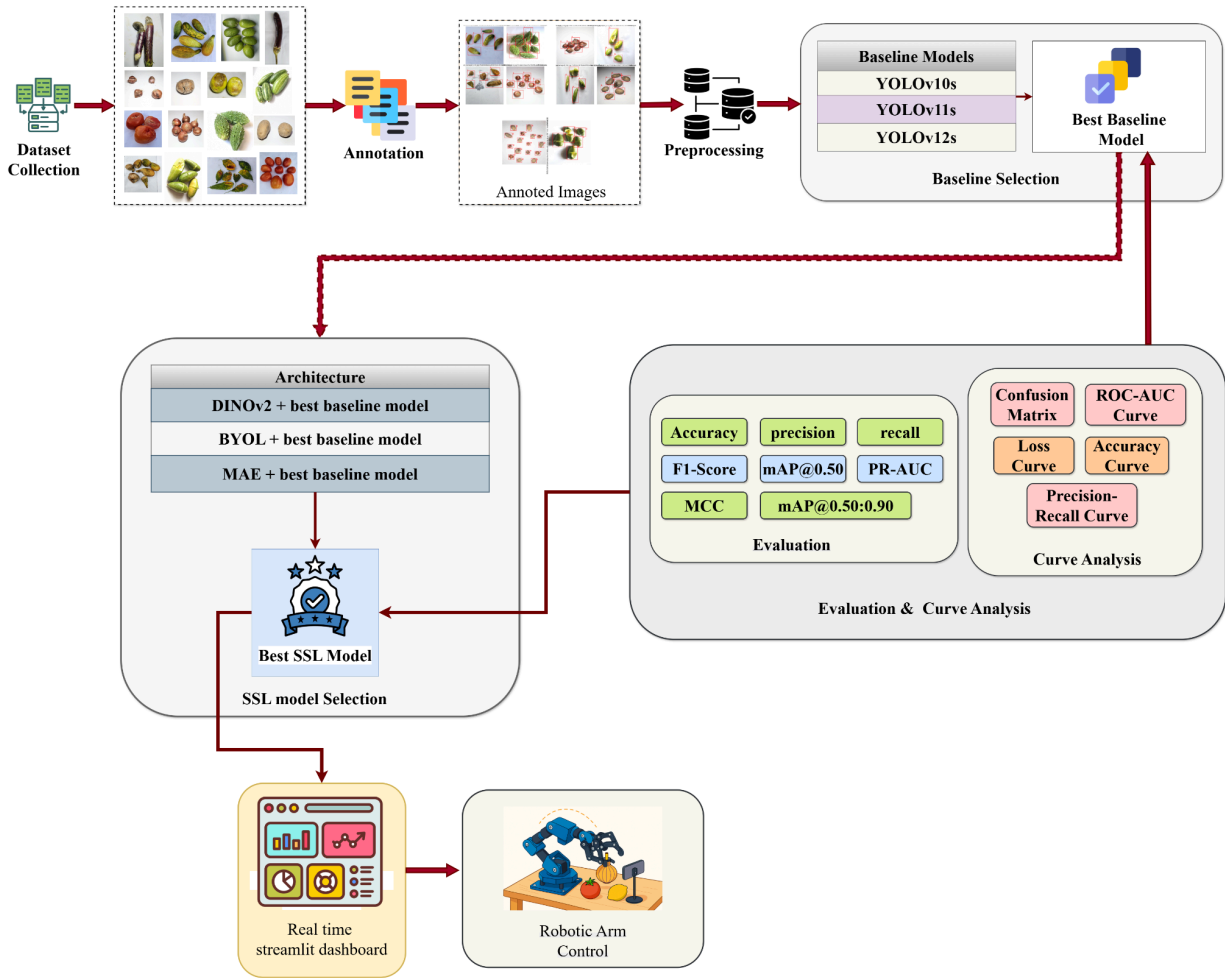
Chai et al. [29] proposed a new model called the Cross-Learning Vision Transformer (CL-ViT) for recognizing plant diseases on crops that were not seen during training. Their model achieved a substantial performance improvement of 23.15% over the previous state-of-the-art model (FF-ViT) on this challenging "zero-shot" task. The main architectural novelty is the use of self-supervised learning (SSL) alongside standard supervised learning, using pretext tasks like predicting image rotation and flip to help the model learn general features that work for both seen and unseen data. A key dataset challenge was the lack of training examples for specific crop-disease combinations; the model was trained on datasets like PlantVillage but tested on unseen pairs. The data was preprocessed with specific augmentations (rotations, flips, patch randomization) designed to teach the model general disease symptoms rather than memorizing specific images. This work addresses the critical research gap that existing models fail when faced with new combinations of plants and diseases not present in the training set. By using SSL to learn a more generalized feature space, the CL-ViT model significantly reduces the performance gap between seen and unseen classes, making plant disease recognition systems more practical for real-world use.

Khoiruddin et al. [30] developed a fruit and vegetable classification system using the MobileNetV2 model, a type of Convolutional Neural Network (CNN). Their model achieved a high accuracy of 97.31%

on a dataset containing 36 different classes. The main architectural choice was using the pre-trained MobileNetV2 architecture, which is known for being efficient and fast, making it suitable for applications on mobile devices. A key challenge with the dataset was class imbalance, where some types of fruits and vegetables had more images than others, which sometimes caused misclassifications (e.g., an apple being mistaken for a chili). The data was preprocessed by resizing all images to 224x224 pixels and normalizing the RGB color values to a range of 0-1. This work addresses the research gap for a fast and accurate automated system that can classify a wide variety of fruits and vegetables, which is useful for tasks like inventory counting in retail. The high accuracy demonstrates that efficient models like MobileNetV2 are very effective for this kind of image classification task, even with a diverse set of classes.

## Methodology

The overall working procedure of this project in Figure 1 involves a sequential pipeline which starts by understanding the dataset. The dataset was manually collected within 8 different vegetables, 16 subclasses and one multilabel that includes other 16 subclasses with caution and then annotated with a roboflow annotator. The dataset is separated in train, test and valid subsets. After that, we will preprocess the dataset to resolve any corrupt image, misclassification, false annotations and the augmentation and then an object detection model's three different versions will be applied which includes YOLOv12, YOLOv11, YOLOv10 model to find the best performing backbone and XAI for interpretation. Then, will apply three Self Supervised Learning methods to analyze and validate the models performance through evaluation, clustering and loss curve analysis. Also, using the best performing SSL method, we will build a streamlit web application dashboard over the SSL models and detection functionalities for vegetables. Finally, a robotic arm will be built to create a fully automated system to perform pick-and-place whenever a particular vegetable is in range of a standby mobile camera.



**Figure 1: Workflow Diagram**

## 1. Dataset Collection SetUp

The vegetable dataset was obtained using four different Android phones Google Pixel 9, Samsung Galaxy S23, Vivo Y22s, Apple iPhone 15 Pro Max, and Samsung Galaxy A55 to ensure diversity in imaging conditions and quality of capture. The images were captured in front of a big white art paper background to ensure comparable contrast and minimize visual noise. The vegetables were posed both sparsely and in a structured fashion to incorporate spatial variability, and images were taken in two visual states: fresh and damaged. This setup allowed controlled but varied image acquisition, enhancing the dataset's robustness and generalizability to future multi-class classification and computer vision tasks.

### Mobile Device Specifications

#### 1. Google Pixel 9

- Display: 6.3-inch AMOLED, 1080 x 2400 pixels
  - Rear Camera: 50 MP wide, 12 MP ultrawide
  - Front Camera: 10.8 MP
  - Processor: Google Tensor G3
  - RAM: 8 GB
  - Storage: 128/256 GB
2. **Samsung Galaxy S23**
- Display: 6.1-inch Dynamic AMOLED 2X, 1080 x 2340 pixels
  - Rear Camera: 50 MP wide, 12 MP ultrawide, 10 MP telephoto
  - Front Camera: 12 MP
  - Processor: Snapdragon 8 Gen 2
  - RAM: 8 GB
  - Storage: 128/256 GB
3. **Vivo Y22s**
- Display: 6.55-inch IPS LCD, 720 x 1612 pixels
  - Rear Camera: 50 MP wide, 2 MP macro
  - Front Camera: 8 MP
  - Processor: MediaTek Helio G85
  - RAM: 4 GB
  - Storage: 128 GB
4. **Samsung Galaxy A55**
- Display: 6.4-inch Super AMOLED, 1080 x 2400 pixels
  - Rear Camera: 64 MP wide, 12 MP ultrawide, 5 MP macro, 5 MP depth
  - Front Camera: 32 MP
  - Processor: Exynos 1380
  - RAM: 6 GB
  - Storage: 128 GB
5. **Apple iPhone 15 Pro Max**
- Display: 6.7-inch Super Retina XDR OLED (2796×1290 pixels, ~460 ppi) with ProMotion (up to 120 Hz), Always-On display.
  - Rear Camera: 48 MP wide (main) + 12 MP ultrawide + 12 MP telephoto (5× optical zoom on Pro Max)
  - Front Camera: 12 MP wide
  - Processor: Apple A17 Pro chip (3 nm)
  - RAM: 8 GB
  - Storage options: 256 GB

## 2. Dataset Collection Strategy

To develop an effective multi-class vegetable classification and detection dataset, we collected images of eight types of non seasonal vegetables like cucumber, tomato, bitter gourd, potato, pointed gourd, onion, eggplant, and lemon in two visual conditions: fresh and damaged. This created sixteen sub-classes, each represented in an orderly manner to ensure balance within the dataset. Table 1 shows the dataset collection distribution for this work.

At First, Single-class images were captured first to represent individual vegetable conditions on their own. 50 images per sub-class were collected, totaling 800 single-class images. There was only one instance of the vegetable per image, photographed against controlled lighting and neutral backgrounds to minimize visual noise. Afterwards, to promote intra-class variation, single-class combinational images were collected in scattered and structured arrangements. 105 scattered and 105 structured images were captured for each sub-class, where multiple instances of the same sub-class were randomly or structurally arranged, respectively. These images represented spatial distribution variation while retaining single-class purity, yielding 210 combinational images per sub-class.

Then, the multi-label subset was prepared to simulate real-world classification and detection problems with co-occurring vegetable types. Vegetables were paired in four pre-decided pairs: (tomato + cucumber), (bitter gourd + pointed gourd), (potato + onion), and (brinjal + lemon). There were images of fresh and damaged conditions of both the vegetables in each pair, thus four sub-classes for each pair. For each pair, 120 multi-label images were collected, divided evenly into 60 dispersed and 60 structured configurations. In every multi-label image, all four sub-classes within a pair co-occurred together. To add variation, dominance was switched between sub-classes within a pair: some images had one sub-class in higher numbers (five to six instances) and the others in lower numbers (two to three instances).

Finally, the total dataset comprises 4,640 images in all collection methods in a perfectly balanced manner across all 16 sub-classes. Each vegetable pair equally contributed to the multi-label subset with 120 images per pair. This arrangement allows for both intra-class variability (fresh vs damaged) and inter-label challenge (multi-label co-occurrence), making the dataset suitable to train robust classification and detection models with high accuracy.

**Table 1: Data Collection Strategy Distribution Table**

| <b>Dataset Category</b>                | <b>Subset / Pair</b>         | <b>Per Sub-Class</b> | <b>Total Sub-classes</b> | <b>Arrangement</b>                     | <b>Total Images</b> |
|----------------------------------------|------------------------------|----------------------|--------------------------|----------------------------------------|---------------------|
| <b>Single-class</b>                    | Isolated                     | 50                   | 16                       | Individual vegetables                  | 800                 |
| <b>Single-class Combinationa<br/>I</b> | Scattered                    | 105                  | 16                       | Random placement of multiple instances | 1,680               |
| <b>Single-class Combinationa<br/>I</b> | Organized                    | 105                  | 16                       | Structured rows/clusters               | 1,680               |
| <b>Multilabel</b>                      | Tomato + Cucumber            | 120                  | 4                        | 60 Scattered + 60 Organized            | 120                 |
| <b>Multilabel</b>                      | Bitter Gourd + Pointed Gourd | 120                  | 4                        | 60 Scattered + 60 Organized            | 120                 |

|                   |                     |     |   |                                |       |
|-------------------|---------------------|-----|---|--------------------------------|-------|
| <b>Multilabel</b> | Potato + Onion      | 120 | 4 | 60 Scattered +<br>60 Organized | 120   |
| <b>Multilabel</b> | Eggplant +<br>Lemon | 120 | 4 | 60 Scattered +<br>60 Organized | 120   |
| <b>Total</b>      |                     |     |   |                                | 4,640 |

### 3. Dataset Description

After the strategy, we tried to collect the same number of images. Due to some image file error, the roboflow removed some corrupted image files. For this work, the dataset is well balanced for both classification and annotated detection type analysis. The classification images are divided into single class and multiclass images. The classification single class images are distributed in 8 different classes where each has two types of subclass (Fresh and Damaged) shown in Table 2 and multilabel images in quadrates in a single folder with a total of 541 images. Finally, the annotated images for detection are distributed into train 70%, test 15% and validation 15% merging both single class and multi-labels images.

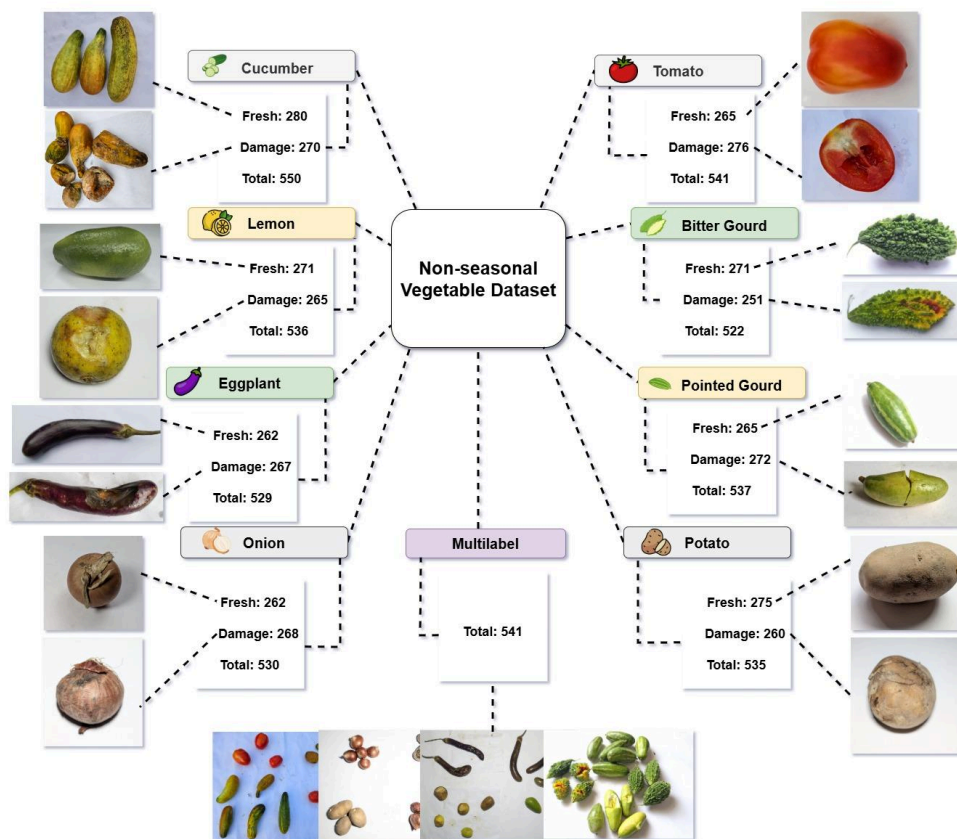
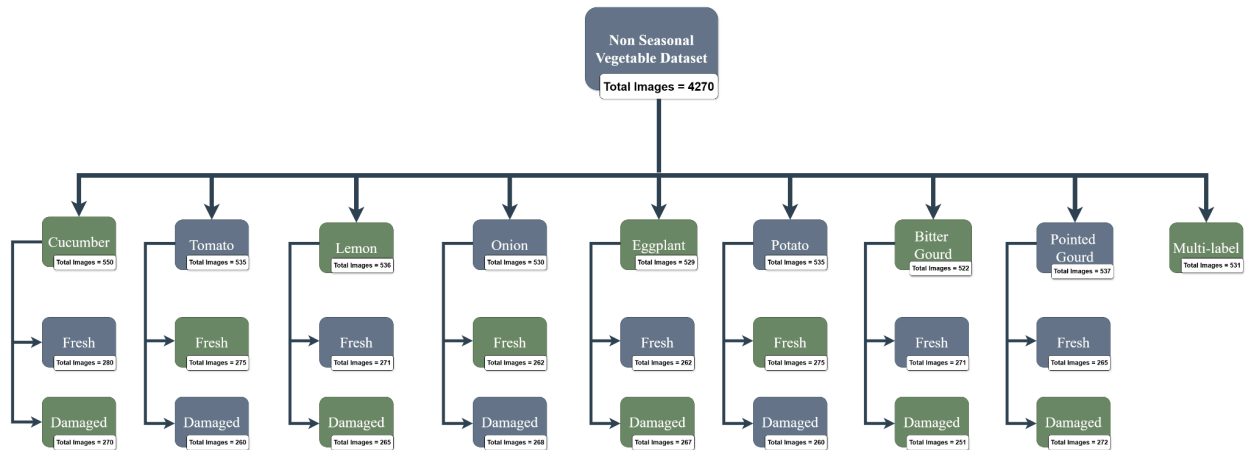


Figure 2 : Dataset Workflow

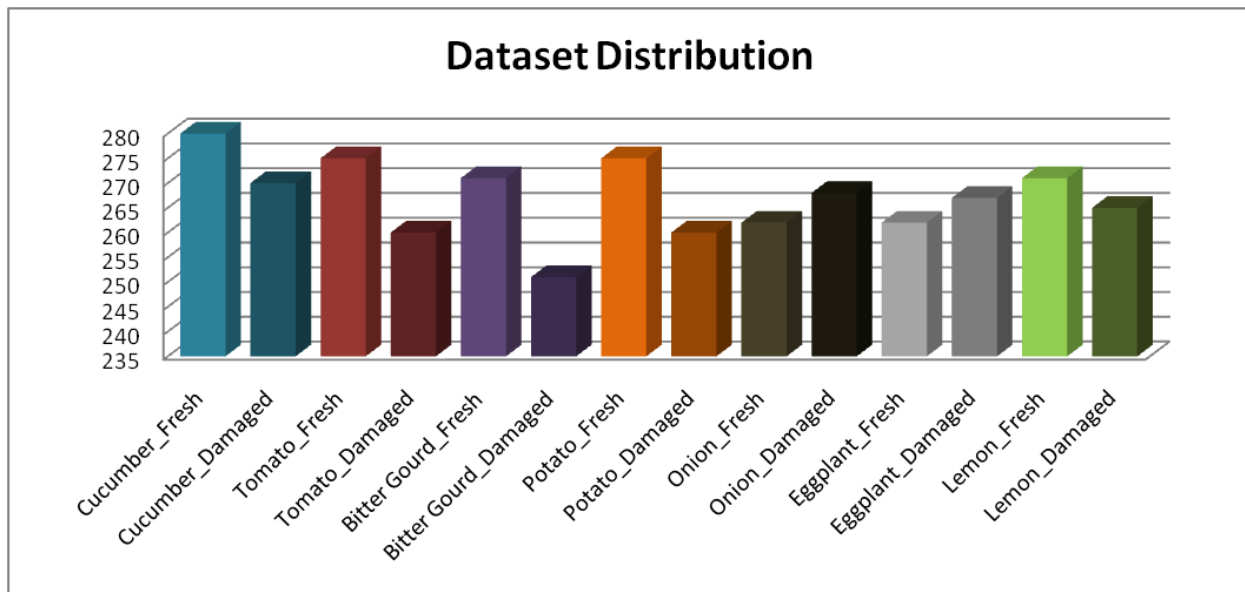
The single class image number is shown in Table 2, Figure 2 and Figure 3 which are more or less almost the same in numbers like the strategy that has been made. After taking more or less the same number of images per subclass, that makes 4270 in total. Then we got 531 in the total number of multilabel images. Moreover, Figure 4 and Figure 5 shows the histogram of the dataset to visualize that the dataset is well balanced throughout the distribution and some dataset visualizations..



**Figure 3: Classwise distribution Flow**

**Table 2: Dataset distribution for Single Class Classification images**

| Class         | Fresh | Damaged | Total |
|---------------|-------|---------|-------|
| Cucumber      | 280   | 270     | 550   |
| Tomato        | 275   | 260     | 535   |
| Bitter Gourd  | 271   | 251     | 522   |
| Pointed Gourd | 265   | 272     | 537   |
| Potato        | 275   | 260     | 535   |
| Onion         | 262   | 268     | 530   |
| Eggplant      | 262   | 267     | 529   |
| Lemon         | 271   | 265     | 536   |
| Total         |       |         | 4270  |



**Figure 4: Dataset Distribution for Object Detection**



**Figure 5: Visualized Images**

## 4. Dataset Annotation and Preprocessing

After the collection, all collected images were standardized before training to ensure standard quality, filter between the healthy and corrupted images, augmentation to improve model performance. Each image was resized to 640x640 pixels. One simple augmentation technique will be applied named horizontal flip because the image is collected in a way that no other augmentation is required to train the model. After that the dataset was divided into 70% training, 15% validation and 15% testing sets, ensuring class balance for both fresh and damaged vegetable samples after the annotation on roboflow.

In addition to preprocessing, the dataset was annotated collaboratively using the Roboflow platform. Each participant took on specific annotation responsibilities, allowing the work to be divided fairly and ensuring that the labels remained consistent across all categories. The annotation process focused on bounding box labeling for both fresh and damaged vegetable samples. The overall contributions are presented in Table 3, which records how the tasks were distributed among participants and reflects the collective effort behind preparing the dataset. This organized approach not only made the annotation process more efficient but also provided clarity and transparency regarding each participant's role. After the annotation, the dataset version was formatted into YOLO(images and labels) shown in Figure 6.

**Table 3: Distribution of Dataset Annotation**

| No. | Contributors         | Class                 | Numbers Of Images |
|-----|----------------------|-----------------------|-------------------|
| 1   | Fathhur Rahaman Sams | Bitter Gourd Fresh    | 271               |
|     |                      | Bitter Gourd Damaged  | 251               |
|     |                      | Pointed Gourd Fresh   | 265               |
|     |                      | Pointed Gourd Damaged | 272               |
|     |                      | Multilabeled          | 135               |
| 2   | Sanjana Kazi Supti   | Tomato Fresh          | 265               |
|     |                      | Tomato Damaged        | 271               |
|     |                      | Cucumber Fresh        | 280               |
|     |                      | Cucumber Damaged      | 270               |
|     |                      | Multilabeled          | 132               |
| 3   | Md Junaeid Ali       | Potato Fresh          | 273               |
|     |                      | Potato Damaged        | 260               |
|     |                      | Onion Fresh           | 262               |

|   |                  |                  |     |
|---|------------------|------------------|-----|
| 4 | Mahfuj Alam Imon | Onion Damaged    | 268 |
|   |                  | Multilabeled     | 132 |
|   |                  | Eggplant Fresh   | 262 |
|   |                  | Eggplant Damaged | 267 |
|   |                  | Lemon Fresh      | 264 |
|   |                  | Lemon Damaged    | 265 |
|   |                  | Multilabeled     | 132 |
|   |                  |                  |     |

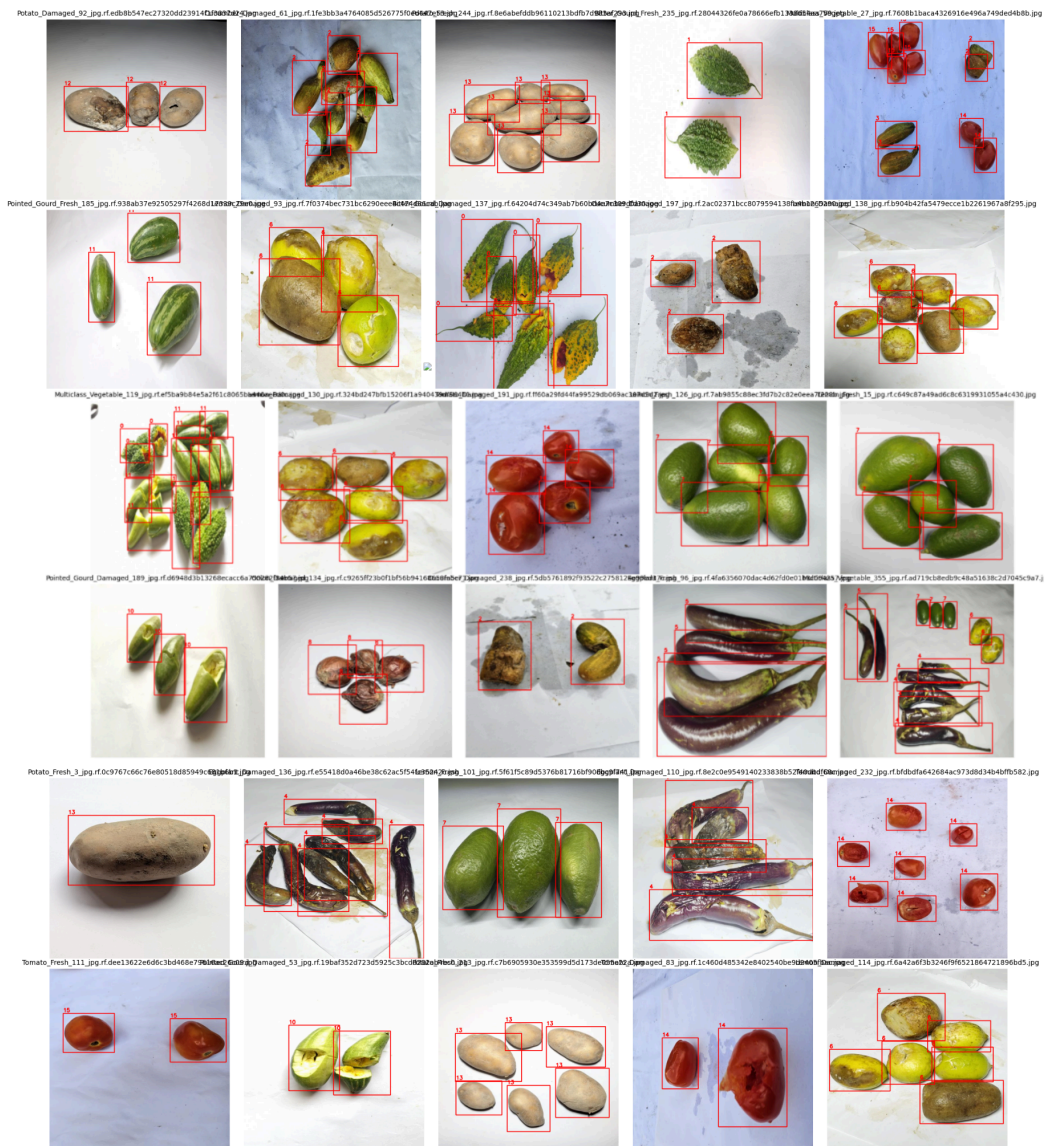


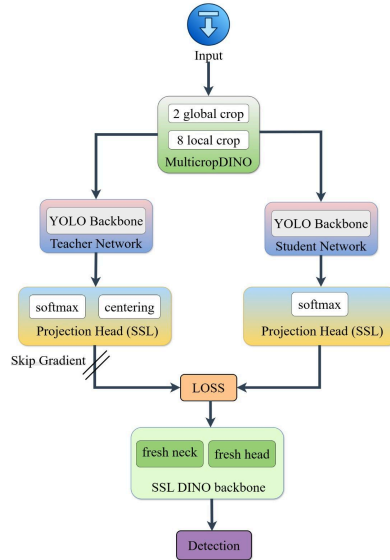
Figure 6: YOLO Formatted Annotated Images

## 5. Supervised Models

To establish a robust baseline for real-time object detection in our automated vegetable sorting system, we will utilize three state-of-the-art supervised models: YOLOv10, YOLOv11, and YOLOv12. These models are selected for their proven efficacy in object detection tasks, particularly in detecting and localizing objects with high precision and speed, which is critical for our robotic arm to accurately identify and sort vegetables into designated bowls or baskets. Our self-curated dataset comprises eight distinct vegetable classes which are cucumber, tomato, bitter melon, potato, pointed melon, onion, eggplant, lemon and one multiclass category for mixed or ambiguous objects, totaling approximately 4,797 images with approximately 550 images per class, 250+ per subclass and 541 multilabel images. Each image is annotated with bounding boxes and class labels to support object detection tasks. Training will leverage transfer learning by initializing with pre-trained weights from the COCO dataset transferred to YOLO, adapting the models to our specific vegetable detection task. The YOLO models will be optimized using the AdamW optimizer with a cosine annealing learning rate scheduler to ensure stable convergence. The loss function will combine localization loss (for bounding box accuracy), confidence loss, and classification loss (for class and multiclass detection), targeting best mean average precision (mAP50) and an intersection over union (IoU) score above 0.7. Data augmentation techniques, including random scaling, shearing, and color jittering, will be applied to enhance model robustness to variations in lighting, orientation, and object appearance. Performance will be evaluated on a held-out validation set using metrics such as mAP50, precision, recall, and F1-score, allowing comparison across YOLOv10, YOLOv11, and YOLOv12 to establish a benchmark for object detection performance.

## 6. SSL Architecture

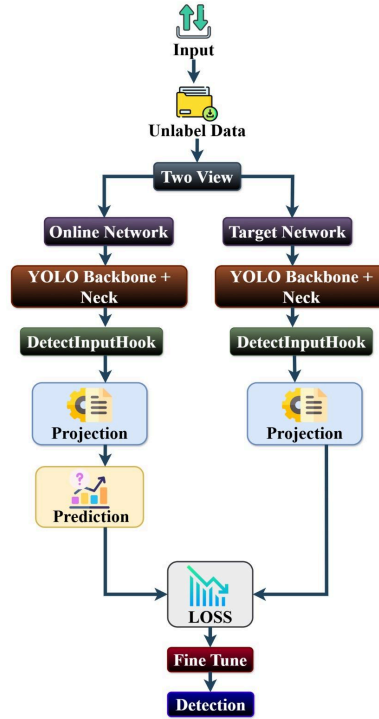
In an effort to address possible limitations in the availability of labeled data and enhance object detection resilience, we shall implement self-supervised learning (SSL) architectures in the guise of DINO (Self-Distillation with No Labels), BYOL (Bootstrap Your Own Latent), and MAE (Masked Autoencoder). These architectures are selected for their proven ability to learn expressive, transferable feature representations from unsupervised images, and this is particularly valuable to us for our preprocessed dataset of 4,797 images of vegetables that appear with different appearances, shapes, and textures. With pre-training on the unlabeled portion of this dataset, these SSL methods are expected to improve detection accuracy, generalization, and robustness, enabling the robotic arm to detect and sort vegetables accurately in real time.



**Figure 7: DINOv2 Style Architecture**

The DINO architecture shown in Figure 7 will utilize a student-teacher network architecture so that the student network will learn to synchronize its feature representations with those of a momentum-updated teacher network across various image augmentations. This self-distillation process enables the model to learn discriminative and invariant features such as vegetable outlines, color transitions, and surface textures crucial for accurate bounding box prediction and class recognition. The DINO model will be pre-trained on the unlabeled segment of our dataset and then fine-tuned on labeled data to optimize object detection in the form of accurate localization and classification in eight categories of vegetables and a multiclass category.

On the other hand, the BYOL architecture in Figure 8 will be employed to learn visual representations through predicting the output of a target network from differently augmented views of the same image, eliminating negative pairs. This approach is computationally efficient and suitable to our dataset size. The pretrained BYOL encoder will then be fine-tuned to the downstream object detection task for it to output accurate bounding boxes as well as class predictions. The best BYOL model will aid the robotic arm in its operation by providing robust detections that feed into pick-and-place protocols, placing each vegetable into its own bowl or basket.



**Figure 8: BYOL Architecture**

Additionally, we will integrate the MAE (Masked Autoencoder) structure in order to further support representation learning. MAE functions by masking a significant percentage (75%) of input image patches and learning a yolo encoder-decoder model to complete the missing information. This encourages the encoder to focus on global semantic structures and not just local pixel similarities, enabling it to learn context-dependent, shape- and texture-dependent embeddings needed to distinguish visually similar vegetables. The MAE-pretrained encoder will then be fine-tuned for object detection, with increased resilience to occlusions, changes in lighting conditions, and background noise.

All three SSL-pretrained models (BYOL, DINO, and MAE) will be evaluated relative to supervised YOLO baselines on standard detection metrics including mAP@0.5, mAP@0.5:0.9, IoU, precision, recall, and F1-score. We anticipate SSL pre-training to substantially influence detection performance, particularly in challenging conditions with limited labeled samples or visual ambiguity. The highest-performing SSL model will be integrated in the real-time robotic sort pipeline, which will talk to the robotic arm through the Robot Operating System (ROS). Detection output will drive precise pick-and-place actions, and real-time detection output will be streamed to the web dashboard for real-time monitoring of the sort.

## 7. Experimental Setup and Training Configuration

All experiments were conducted under a uniform training regimen to ensure fair comparison across different configurations. Training was performed on the Kaggle GPU environment equipped with an NVIDIA Tesla T4 (15 GB VRAM), 4 CPUs, and 31 GB RAM, using PyTorch 2.6.0 with CUDA 12.4 support and the Ultralytics YOLO framework (v8.3.232). The vegetable freshness dataset contained sixteen categories, representing fresh and damaged states across multiple vegetable types, and all images were resized to 640×640 pixels before training. Two paradigms were examined: self-supervised pipelines (DINOv2-style and BYOL encoders) and supervised YOLO baselines (YOLOv10-s, YOLOv11-s, YOLOv12-s). Pretrained SSL weights were reused to initialize YOLO detectors, ensuring fair comparison. Training was conducted for 50 epochs with a batch size of 12, learning rate 0.01, momentum 0.937, and weight decay 0.0005, followed by a 3-epoch warmup where bias learning rate was set to 0.1 and momentum gradually increased to 0.8. Detection employed a confidence threshold of 0.25 and IoU thresholds of 0.50–0.95, while SSL models fixed the temperature parameter at 0.1 for stable similarity estimation.

Table 4 presents the supervised YOLO baselines. All three models adopt AdamW (auto) as the optimizer, with identical batch sizes (12) and image sizes (640×640). Training spans 50 epochs, with learning rates automatically tuned around 0.002. Weight decay is consistently set at 0.0005, and a warmup of 3 epochs is applied. Momentum remains fixed at 0.9, with no gradient accumulation. Structural differences emerge in hidden dimensions: YOLOv12-s integrates approximately 464 fused layers, YOLOv11-s uses 203 layers, while YOLOv10-s remains model-defined. Despite these variations, all three models employ BCE + DFL for layer definitions, disable explicit loss functions (marked as False), and support multi-scale training and mixed precision (AMP). Pretrained weights differ only in version-specific checkpoints (yolo12s.pt, yolo11s.pt, yolo10s.pt). Analytically, these similarities underscore the stability of YOLO’s training pipeline, while the differences in hidden dimensions reflect incremental architectural refinements across versions.

**Table 4: Comparative Hyperparameter Settings for Baseline Models**

| Hyperparameter    | YOLOv12-s    | YOLOv11-s    | YOLOv10-s    |
|-------------------|--------------|--------------|--------------|
| <b>Optimizer</b>  | AdamW (auto) | AdamW (auto) | AdamW (auto) |
| <b>Batch Size</b> | 12           | 12           | 12           |
| <b>Image Size</b> | 640          | 640          | 640          |
| <b>Epochs</b>     | 50           | 50           | 50           |

|                              |                            |                      |               |
|------------------------------|----------------------------|----------------------|---------------|
| <b>Learning Rate</b>         | auto (~0.002)              | auto (~0.002)        | auto (~0.002) |
| <b>Weight Decay</b>          | 0.0005                     | 0.0005               | 0.0005        |
| <b>Warm-Up</b>               | 3 epochs                   | 3 epochs             | 3 epochs      |
| <b>EMA / Momentum</b>        | Momentum=0.9               | Momentum=0.9         | Momentum=0.9  |
| <b>Gradient Accumulation</b> | 0                          | 0                    | 0             |
| <b>Hidden Dim / Channels</b> | $\approx 464$ fused layers | $\approx 203$ layers | Model-defined |
| <b>Number of Layers</b>      | BCE + DFL                  | BCE + DFL            | BCE + DFL     |
| <b>Loss Function</b>         | False                      | False                | False         |
| <b>Multi-Scale Training</b>  | True                       | True                 | True          |
| <b>Mixed Precision (AMP)</b> | True                       | True                 | True          |
| <b>Pretrained Weights</b>    | yolo12s.pt                 | yolo11s.pt           | yolo10s.pt    |

Table 5 outlines the distinct design choices between the two self-supervised pipelines. DINOv2 employs a larger hidden dimension (2048) and Softmax activation, reflecting its emphasis on high-capacity feature extraction, while BYOL uses a smaller hidden dimension (512) with ReLU, favoring efficiency. DINOv2 integrates teacher and student temperature parameters (0.07 and 0.1) to stabilize similarity estimation, whereas BYOL omits these but applies a stricter confidence threshold (0.7). Both share commonalities such as AdamW optimizer, 50 epochs, and EMA = 0.996, yet their divergence in loss functions Cross-Entropy for DINOv2 versus Mean Squared Error for BYOL illustrates differing philosophies in representation learning. These contrasts highlight how DINOv2 prioritizes structured feature alignment, while BYOL emphasizes smoother embedding consistency.

**Table 5: Comparative Hyperparameter Configuration across SSL Architectures**

| <b>Hyperparameter</b>       | <b>DINOv2 Style</b> | <b>BYOL</b> |
|-----------------------------|---------------------|-------------|
| <b>Optimizer</b>            | AdamW               | AdamW       |
| <b>Backbone Batch Size</b>  | 32                  | 64          |
| <b>Fine-Tune Batch Size</b> | 8                   | 8           |
| <b>Feature Image Size</b>   | 224                 | 165         |
| <b>Fine-Tune Image Size</b> | 640                 | 640         |

|                             |               |                    |
|-----------------------------|---------------|--------------------|
| <b>Grad. Accumulation</b>   | 2             | -                  |
| <b>Activation Function</b>  | Softmax       | RelLu              |
| <b>Hidden Dimension</b>     | 2048          | 512                |
| <b>Epochs</b>               | 50            | 50                 |
| <b>Confidence Threshold</b> | -             | 0.7                |
| <b>Teacher Temperature</b>  | 0.07          | -                  |
| <b>Student Temperature</b>  | 0.1           | -                  |
| <b>Learning Rate</b>        | 5e-4          | 1e-3               |
| <b>Weight Decay</b>         | 0.05          | 1e-4               |
| <b>Loss Function</b>        | Cross-Entropy | Mean Squared Error |
| <b>EMA</b>                  | 0.996         | 0.996              |
| <b>Warmup Fraction</b>      | 0.1           | -                  |
| <b>Centering Momentum</b>   | 0.9           | -                  |

To improve generalization, RandAugment was applied during SSL pretraining, and YOLO fine-tuning incorporated mosaic augmentation, HSV jitter (h=0.015, s=0.7, v=0.4), random translation (0.1), random erasing (0.4), and horizontal flips (p=0.5). These augmentations enhanced robustness against lighting, occlusion, and background clutter. Final detectors were evaluated using the best validation checkpoint, ensuring stable and reliable performance reporting.

## 8. Evaluation Matrices

To maintain consistency and rigor in performance assessment, this study utilized a set of evaluation metrics that collectively address both classification accuracy and localization precision—key factors in agricultural object detection, particularly for freshness classification. Given the challenge of distinguishing between healthy and damaged vegetable states, the following metrics were selected:

- Precision:** Precision reflects the proportion of correctly predicted fresh or damaged vegetable instances out of all positive predictions. In freshness detection, this metric helps reduce false positives, such as misclassifying background textures or non-vegetable elements as produced. It plays a vital role in downstream tasks like automated sorting and shelf-life estimation. The formula is:

$$\text{Precision} = \frac{TP}{TP+FP}$$

where  $TP$  and  $FP$  denote true positives and false positives, respectively.

- **Recall:** Recall indicates how effectively the model identifies actual instances of fresh or damaged vegetables. A high recall ensures minimal false negatives, which is crucial for applications like spoilage tracking or yield analysis. The metric is defined as:

$$\text{Recall} = \frac{TP}{TP+FN}$$

with  $FN$  representing false negatives.

- **Mean Average Precision (mAP):** To evaluate detection performance across varying confidence thresholds, two mAP variants were reported:

—**mAP@50:** Measures average precision at a fixed IoU threshold of 0.5. This relatively lenient metric is widely used in agricultural detection benchmarks. It often yields higher scores and serves as a common baseline in agricultural detection studies. In freshness detection tasks, mAP@50 helps capture general detection capability without heavily penalizing minor localization errors.

—**mAP@[50:90]:** Computes average precision across multiple IoU thresholds (from 0.5 to 0.9 in 0.05 increments), offering a more stringent assessment of localization accuracy. It gives a stricter way to check how well models match object boundaries. For vegetable freshness detection, mAP@[50:90] is especially useful because it tests the model's strength under tougher conditions. This helps in spotting small or subtle damaged areas more reliably.

- **Intersection over Union (IoU):** IoU serves as the core criterion for bounding box evaluation, quantifying the overlap between predicted and ground truth annotations. It is calculated as:

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area Of Union}}$$

A prediction is considered valid if its IoU exceeds the designated threshold.

## 9. Explainable AI techniques

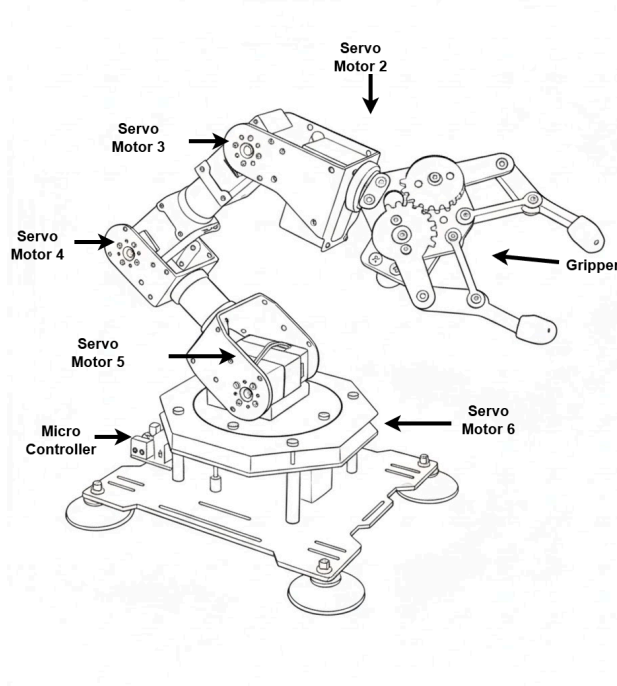
Explainable Artificial Intelligence (XAI) is a method that helps to understand how and why a deep learning model makes its decisions. In this project, two well-known XAI techniques, Grad-CAM (Gradient-weighted Class Activation Mapping) and LIME (Local Interpretable Model-agnostic

Explanations), are used to explain the model behavior for vegetable classification and freshness detection. Grad-CAM creates a heatmap showing which parts of an image the model focuses on during prediction. It highlights key areas like color or damaged spots, helping verify that the model pays attention to correct regions such as rotten parts of vegetables. LIME explains individual predictions by slightly changing the image and observing how the output changes. It shows which features most affect a single result, such as color or texture differences between fresh and spoiled samples. Together, Grad-CAM and LIME make the model more transparent, allowing users to see how and why each sorting decision is made.

## **10. Realtime Web Application Dashboard**

The built real-time web application, on the Streamlit platform, is a web-based interactive and friendly interface for analyzing vegetables. The system enables image-based object detection and video annotation and supports media file uploading to visually detect vegetable instances. An integrated live camera detection feature is supported to enable real-time viewing, best applicable for continuous viewing, experimental testing, or dynamic data capture. Annotated video clips and processed images can be saved to allow the user to store results for further examination. The software also provides dynamic visualization of measurements with live plots to facilitate easier interpretation of results in real-time and improve data-driven decision-making. In addition to the detection and annotation functionalities, a dedicated image classification module is implemented, which utilizes self-supervised learning (SSL) techniques. The module also provides interpretability tools such as Grad-CAM and LIME to facilitate users to understand model decisions and detect salient regions in images. Moreover, the platform also comprises an object detection-based robotic arm control system, which is used to sort vegetables autonomously. Based on prediction of class or sub-class, the robotic arm can pick up and drop vegetables at desired locations, enabling an end-to-end automation pipeline from detection to execution. Overall, the software seamlessly combines real-time detection, classification, visualization, explainability, and automated control, providing a complete tool for research as well as practical agricultural or food-processing uses.

## 11. Robotic Arm Functionalities



**Figure 9: Robotic Pick and Place Arm with Vision**

Figure 9 provides an overview of the mechanical and actuation architecture of the Hiwonder xArm 1S, a 6-DOF robotic manipulator with a servo-driven parallel gripper. The robot integrates six intelligent servomotors, Servos 2-6, and one gripper servo that together contribute to coordinated multi-joint motion for automated pick-and-place operations. The native microcontroller mounted at the base executes low-level servo commands transmitted by the perception-to-control pipeline. Each actuator has a direction-dependent functional behavior that is vital for generating stable motion trajectories:

### **Gripper Servo (End-Effector):**

Controls finger closure, whereby increasing the value of this command will close the grip tighter and vice versa. This guarantees that vegetables in different shapes and firmness will be securely grasped.

### **Servo Motor 2 - Wrist Roll Rotation:**

Controls the wrist–gripper linkage's rotational orientation. Higher servo values induce clockwise rotation about the axis of the wrist, while lower values induce counterclockwise rotation. This allows for the correction of orientation of vegetables before placing them.

### **Servo Motor 3 — Upper Arm Elevation:**

Responsible for raising/lowering the upper link. Higher values raise it, whereas lower values lower it, which is used for changes in vertical reach.

### Servo Motor 4 - Elbow Compensation Mechanism:

Moves inversely to Servo 3 to provide controlled compliance. Higher values move the segment downward, and lower values lift it upward for smooth articulation of the elbow during pickup.

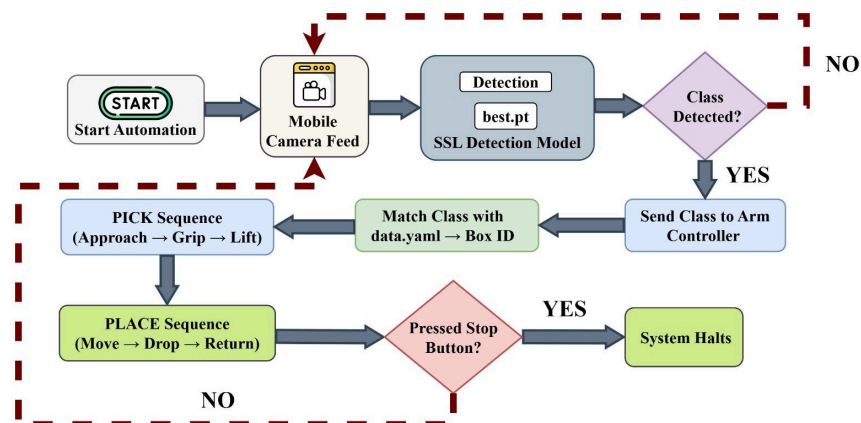
### Servo Motor 5 - Forearm Forward/Backward Translation:

Controls the translational arc movement of the forward arm segment. Increasing values pull the forearm backward and decreasing values push it forward, allowing the end-effector to enter or retract from the workspace.

### Servo Motor 6 — Base Rotation:

Controls the azimuthal movement of the robot. Greater servo values rotate the base left from home and smaller values rotate right. This allows for sector-based navigation to reach class-specific bins.

These servos collectively provide accurate six-degree-of-freedom control, enabling stable approach, grasp, lift, and placement behaviors needed for autonomous manipulation.



**Figure 10: Robotic Functionality Workflow**

The workflow shown in Figure 10 starts with a continuously streaming mobile top-mounted camera of visual data to a Streamlit dashboard where real-time vegetable detection and classification are done by a model trained using SSL, best.pt. When an object is successfully detected by the model, the predicted class label is sent to the robotic arm controller. The arm cross-checks this detected class against the corresponding entry or entries in the predefined class names in the system's data.yaml file and the box locations for sorted placement. Once a match is found, it initiates arm motion just for that particular class of vegetable, ensuring no unnecessary movement without a confirmed detection. It proceeds to execute the pick routine, first approaching the object, positioning the gripper above the detected vegetable, and then performing the grasp. At the end of a successful pickup, the robot goes into a class-specific placement routine, going to the predefined drop location and releasing the item accurately into its container. This whole loop operates autonomously once the user presses the Start Automation button, whereby vegetables are processed sequentially. The system will maintain continuous perception-action

feedback until the Stop button is pressed, after which the robotic arm halts all motion and awaits further commands.

## Results and Discussions

### 1. Evaluation of Supervised Model Performance

The evaluation of the supervised models is shown in Table 6 where the performance of YOLOv10s shows strong detection capability but always remains the least effective of the three variants. In this context, with a precision of 96.9% and recall of 94.2%, the model identifies most objects with a reasonably low number of false positives. The F1-score is 95.5%, showing a balanced harmony between precision and recall. Moreover, with a high value of mAP50 at 95.8%, its performance falls off more noticeably when moving to the more severe IoU thresholds, as presented by its mAP50-95 of 77.4%.

In contrast, YOLOv11s significantly improves nearly all the metrics where precision increases to 97.7%, and recall rises to 96.1%. Compared to YOLOv10s, this model reduces both false positives and false negatives more effectively. Besides, the F1-score equals 96.9%, which suggests a strong balance between precision and recall, allowing it to outperform YOLOv10s in its general reliability. However, YOLOv11s managed to increase its mAP50 to 97.5%, reflecting its enhanced recognition capability. This model reached a more robust mAP50-95 of 81.6%, which indicates a more accurate bounding-box localization under more tight constraints. As a result, YOLOv11s shows an appealing improvement both in the coverage of broad detection and in the spatial precision, thus positioning it as a more optimized and stable model than YOLOv10s.

**Table 6: Supervised Model Performance**

| Model    | Precision | Recall | F1-Score | mAP50 | mAP50-95 |
|----------|-----------|--------|----------|-------|----------|
| YOLOv10s | 96.9%     | 94.2%  | 95.5%    | 95.8% | 77.4%    |
| YOLOv11s | 97.7%     | 96.1%  | 96.9%    | 97.5% | 81.6%    |
| YOLOv12s | 98.8%     | 97.9%  | 98.3%    | 99.2% | 85.7%    |

Finally, YOLOv12s provides the best performance among all models that have been compared and its metrics have continuously outperformed previous versions. With a precision of 98.8% and a recall of 97.9%, the model is outstandingly accurate and complete in detecting objects. This is further established with its F1-score of 98.3%, showing outstanding synergy between finding the true positives and avoiding misclassifications. In addition, YOLOv12s has an excellent mAP50 of 99.2%, which further confirms that it has the best capability of correctly detecting objects at conventional IoU thresholds. Its mAP50-95 is 85.7%, reflecting the strongest fine-grained localization performance and surpassing the marks of YOLOv10s and YOLOv11s by a large margin.

Therefore, continuous improvement in all metrics clearly indicates that YOLOv12s has the most mature optimization, the most stable convergence behavior, and the most precise spatial detection capability, hence becoming the most effective model in this series.

## 2. Evaluation of SSL Model Performance

After the supervised model performance evaluation, this study chooses YOLOv12s as the best performing model to further apply as a Self Supervised Learning backbone to boost the detection performance. For this reason, DINOv2 style and BYOL are chosen as the SSL architecture.

Table 7 summarizes the experimental results, showcasing the performance of BYOL and DINOv2-style pretraining with YOLOv12s detection architecture. First, BYOL-initialized YOLOv12s achieved an excellent mAP50 of 98.4% and mAP50–95 of 83.1%, showing that this model learned solid visual representations without labels. The high precision of 97.6% suggests that this model rarely misclassifies whether vegetables are fresh or damaged, while the recall is 96.8%, which means most of the objects present in images are found. Therefore, BYOL contributes much to stable and reliable performance.

Moreover, the comparison between BYOL and the DINOv2-style approach gives clear results in favor of the latter. The DINOv2-style pretrained YOLOv12s attains an impressive mAP50 of 99.6%, whereas BYOL has a score of over 1.2 percentage points less. Also, the mAP50–95 goes up noticeably to 88.3%, showing that the DINOv2 method helps the network maintain high localization and classification accuracy across more severe IoU thresholds. Similarly, gains in precision (99.1%), recall (98.5%), and F1-score (98.8%) further confirm that the representation learning with the DINOv2-based method yields more balance and capability that is overall superior in detection.

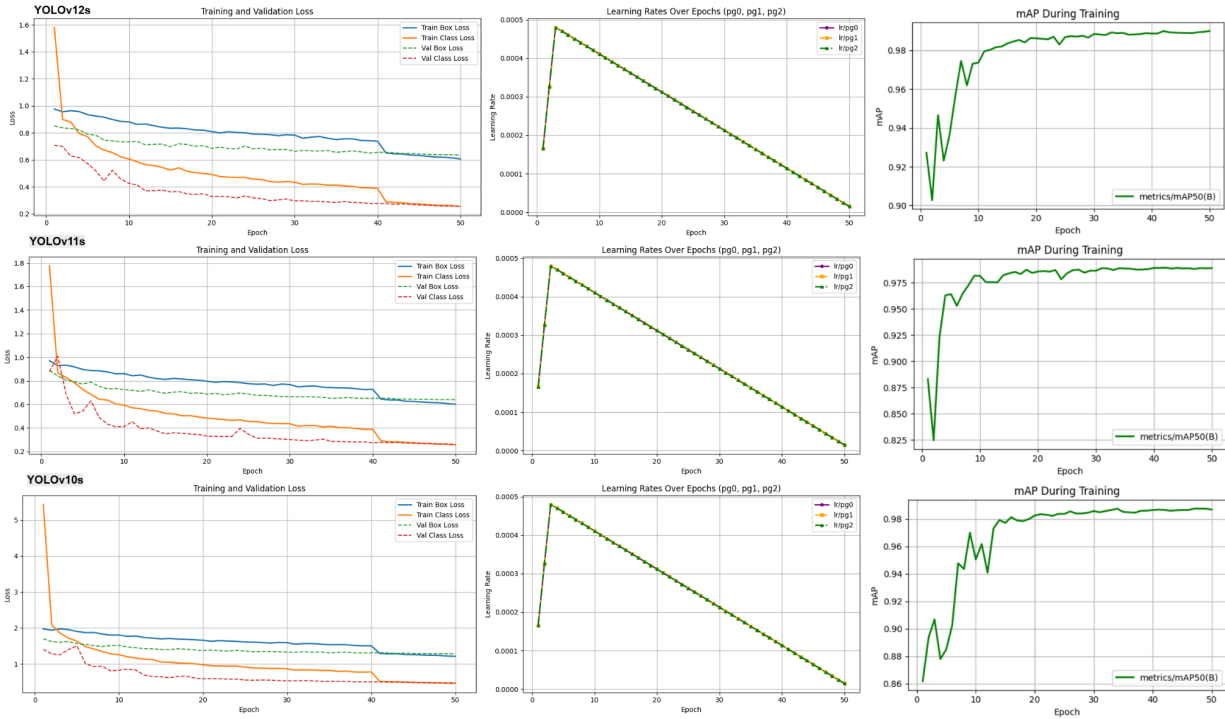
**Table 7: SSL Architecture Performance With Best Baseline Backbone**

| Architecture | Model    | Precision | Recall | F1-Score | mAP50 | mAP50-95 |
|--------------|----------|-----------|--------|----------|-------|----------|
| BYOL         | YOLOv12s | 97.6%     | 96.8%  | 97.2%    | 98.4% | 83.1%    |
| DINOv2 style | YOLOv12s | 99.1%     | 98.5%  | 98.8%    | 99.6% | 88.3%    |

Finally, these gains could be attributed to the architectural and training differences of the two SSL methods. This is specifically because of the richer semantic representations provided by the global-context modeling and enhanced feature invariance in DINOv2, which translate to more accurate bounding box regression and class identification during downstream training. In this respect, while both SSL methods significantly enhance the performance of YOLOv12s compared to baseline supervised learning, DINOv2 clearly proves to be the strongest of the two, providing enhanced performance gains on

all metrics evaluated. These results point to the importance of strong pretraining in enhancing object detection performance, especially within specialized domains such as agricultural produce inspection.

### 3. Loss Curve Performance of Supervised Models



**Figure 11: Loss, Learning and mAP Curve of Baseline YOLO Pretrains**

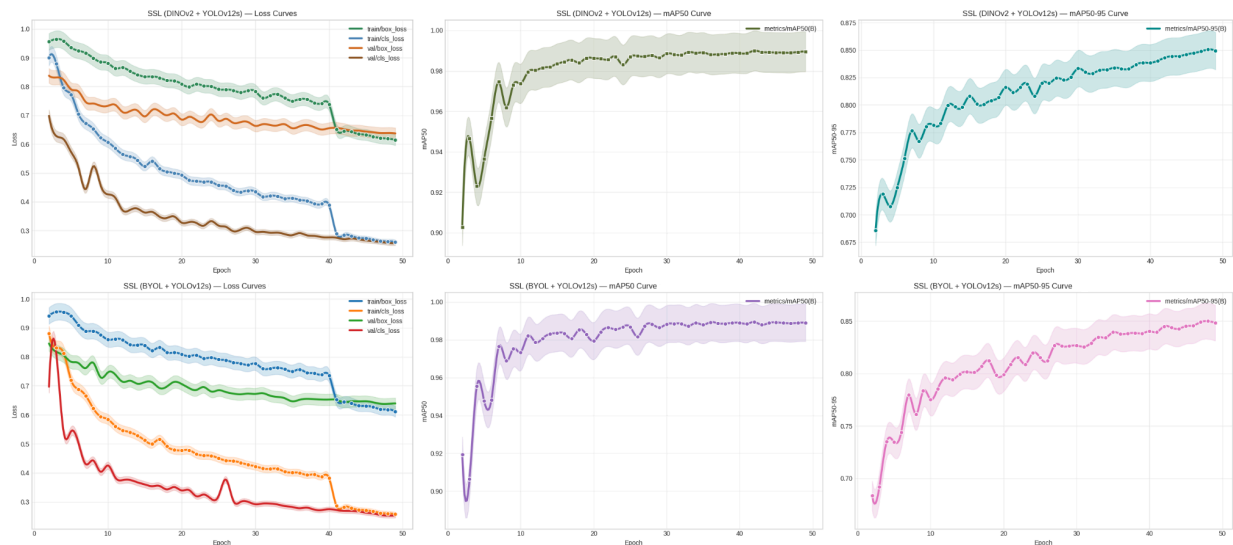
After the evaluation of Supervised models, Figure 11 shows the loss, learning and mAP Curve of Baseline YOLO Pretrains. At First, YOLOv12s showed very stable convergence during training since box and class losses gradually decreased. Even though the losses on validation fluctuated, the general trend was to decrease, which is normal for object detection when objects in images have very different scales. Moreover, consistent optimization behavior was brought about by the nature of the learning rate schedule, it first warmed up and then linearly decayed. Accordingly, the mAP curve increased sharply at the beginning and reached above 0.98 after about epoch 20 then it stabilized. Overall, such results allow claiming that YOLOv12s manages to balance rapid learning, stable convergence, and high accuracy of detection, thus qualifying as a reliable candidate for real-time applications.

Compared to YOLOv12s, the model YOLOv11s showed even smoother and more stable loss patterns, with more uniform decreases in both box and classification losses for the validation set. Besides, although the learning rate schedule was the same across the two models, YOLOv11s seemed to benefit from it more, therefore enjoying more efficient feature extraction in early epochs. Its mAP increased very fast, reaching over 0.97 in the very first several epochs and continuing to go up until reaching a plateau slightly higher than that of YOLOv12s. Therefore, YOLOv11s showed better generalization capability and

slightly better localization precision. This means that YOLOv11s could be especially effective for datasets that require early-stage learning to play an essential role in the downstream detection performance.

Unlike the first two models, YOLOv10s initially showed higher training losses, especially in detection, which indicated a more difficult optimization phase within the first few epochs. However, as the training unfolded, the losses decreased steadily, and the validation curves showed steady improvements, which means it was adapting over time. Moreover, although this model made slower progress at the beginning, the learning rate strategy helped it gradually recover and stabilize. As a result, the mAP curve of the model rises more slowly but eventually approaches values close to 0.98, as for the other models. Hence, even though this YOLOv10s indeed needs more epochs to reach its best performance, it ultimately has very good detection accuracy. In some ways, that implies YOLOv10s suits applications in which longer training time is acceptable in exchange for competitive final results.

#### 4. Loss and mAP Curve Performance of SSL Models



**Figure 12: Loss and mAP Curve of SSL Architectures**

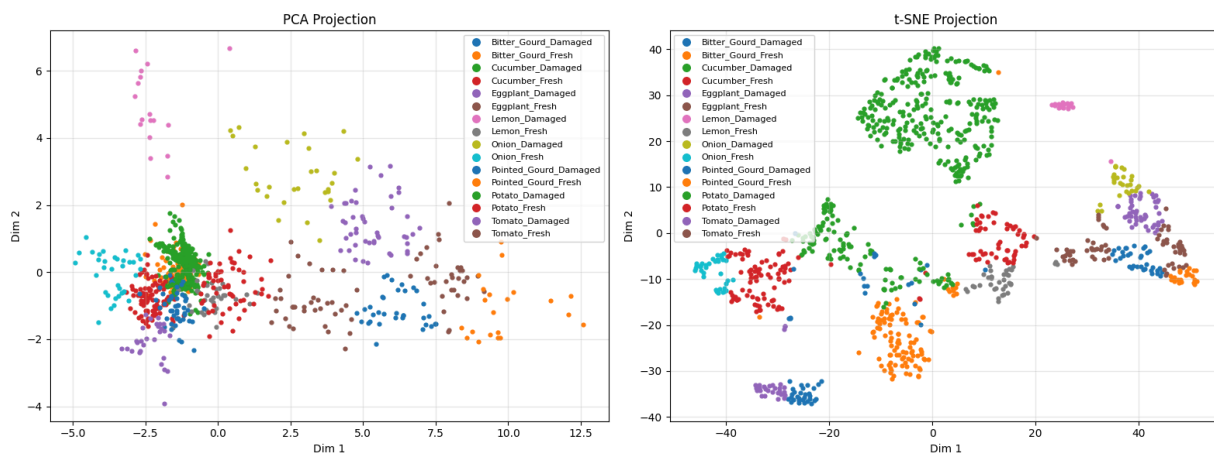
Similar to the supervised model curve analysis, Figure 12 shows the loss and mAP curves after the SSL approach. At First, by incorporating DINOv2, the training behavior of YOLOv12s was very stable, with loss curves steadily decreasing in both the training and validation sets. For example, box and class losses continued to go down with a notable drop at the end of the warmup, whereas their mAP50 rapidly went up during the first few epochs, surpassing 0.96 at an early stage and remaining around 0.99 after that, indicating strong classification and location performance. The mAP50–95 curve also showed progressive improvement linearly throughout its 50-epoch span, which indicates that the model benefited a lot from the representation learning of DINOv2. These are indications that DINOv2 indeed enhanced feature extraction to allow YOLOv12s to converge faster and achieve higher overall accuracy.

In contrast, the BYOL-based SSL pipeline also achieved strong results, while its behavior differed in several important respects. Though initial losses were a little higher and more variable than with the DINOv2 framework, they also steadily decreased through training and converged to similar final values. On the other hand, the mAP50 curve increased steeply in the early epochs, closely following the trend observed in the experiment of DINOv2. However, the peak value fluctuated a bit before converging near 0.98. Correspondingly, the mAP50–95 curve demonstrated a monotonic increase with moderate fluctuations, evidence that BYOL contributed to the fine-grained localization, though in a less stable way than DINOv2. Thus, while BYOL effectively enhanced the generalization capability of the model, the performance gain compared to the DINOv2-enhanced model was only slightly less pronounced and less consistent.

Comparing the two SSL methods side by side, it can be observed that DINOv2 produces a more stable and robust learning signal, which reflects in smoother loss curves and consistently higher mAP gains. BYOL has competitive but slightly less stable behavior, especially for metrics that require fine-scale localization, such as mAP50–95. Both methods significantly outperform the non-SSL baseline, confirming the conclusion above that SSL pretraining notably improves the performance of YOLOv12s on limited or noisy datasets.

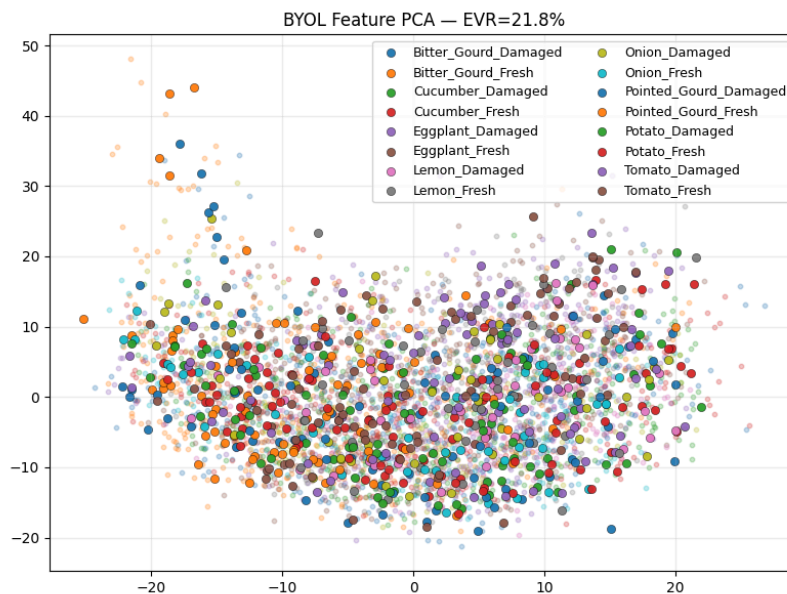
## 5. Clustering Analysis of SSL Models

Figure 13 illustrates the feature space learned by the SSL encoders using PCA and t-SNE visualizations. In the PCA projection, the clusters show considerable overlap, with fresh and damaged vegetables often positioned close together, making separation less clear. On the other hand, the t-SNE projection provides a sharper view, where categories such as cucumber and onion form tight clusters in both fresh and damaged states. In contrast, vegetables like lemon and potato appear more scattered, which may reflect their natural variability in texture and color. Overall, PCA captures only a portion of the variance, while t-SNE reveals more distinct boundaries, highlighting the importance of non-linear methods for analyzing SSL embeddings.



**Figure 13: Clustering of DINOv2 Style Architecture using YOLOv12s**

Figure 14 presents the PCA clustering results for the BYOL encoder, with an explained variance ratio of 21.8%. The plot shows moderate separation, with certain categories such as tomato and potato forming clearer distinctions between fresh and damaged samples. However, overlaps remain for visually similar vegetables, including cucumber and pointed gourd, suggesting that BYOL may struggle when spoilage features are subtle. This spread-out arrangement indicates that BYOL representations capture broader variability, which can be useful for handling diverse appearances but may reduce tight class-specific clustering.

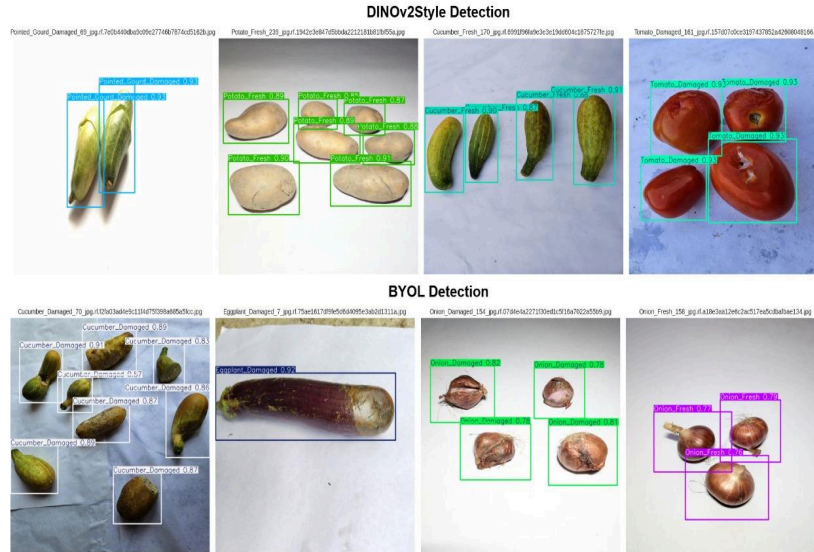


**Figure 14: Clustering of BYOL Architecture using YOLOv12s model**

In summary, the clustering analyses demonstrate complementary strengths of SSL models. DINOv2 produces tighter, more class-focused clusters, supporting accurate freshness classification, while BYOL

generates a wider distribution of features that may improve generalization to new or varied examples. Together, these findings confirm that self-supervised learning can extract meaningful patterns without labels, and they help explain the detection performance observed in earlier experiments. This evidence also points toward future improvements in model design, where combining the strengths of both approaches could enhance agricultural freshness detection systems.

## 6. Detection Visualization of SSL Models



**Figure 15: Vegetable Detection Visualized by SSL Architecture**

The detection performance of the Self-Supervised Learning architectures is validated through annotated images that demonstrate their ability to localize and classify vegetables under varying conditions. Figure 15 compares detection outputs from the DINOv2-style and BYOL architectures, offering qualitative insight into their applicability. The upper section shows results from DINOv2, with bounding boxes tightly aligned and high confidence scores. For example, a damaged eggplant is correctly identified with focus on visible defects, while a fresh pointed gourd is localized without background confusion. These results highlight DINOv2's effective integration of shape, texture, and defect-based features.

The lower section illustrates BYOL detections, which remain accurate but occasionally show positional drift, especially for irregular shapes or overlapping vegetables. In a scene with onion and eggplant, BYOL distinguishes both classes, though some boxes appear oversized. This indicates strong feature learning but slightly weaker localization in cluttered settings. Both models perform well in distinguishing fresh from damaged samples. A damaged cucumber, for instance, is correctly identified despite partial occlusion, confirming robustness across varied lighting and orientations critical for agricultural sorting lines.

Overall, the visualizations confirm that both SSL methods enable accurate detection without labeled supervision. DINOv2 achieves tighter bounding box fits and higher confidence in defect localization, while BYOL provides reliable classification with minor localization variance. These findings align with earlier quantitative metrics and reinforce the feasibility of self-supervised features for automated, vision-guided sorting in dynamic environments.

## 6. Confusion Matrix Analysis

Figure 16 presents the confusion matrix generated using the BYOL feature extractor with a k-NN classifier on the validation set. The matrix shows strong diagonal dominance, confirming that most predictions align with true labels. For instance, Cucumber Fresh achieved 43 correct predictions, Onion Damaged 38, and Tomato Fresh 33, demonstrating the model's ability to capture visual cues such as texture, color, and shape. Misclassifications, however, highlight areas for refinement. Eggplant Fresh was often mistaken for Eggplant Damaged (16 cases), likely due to natural skin variations or lighting effects. Similarly, Tomato Fresh was misclassified as Tomato Damaged in 9 instances, reflecting the difficulty of detecting subtle spoilage cues. Cross-category confusions also occurred, such as Bitter Gourd Damaged vs. Pointed Gourd Damaged (7 cases each), and Potato Fresh misclassified as both Potato Damaged and Onion Fresh, suggesting overlapping visual traits. Overall, most errors occur within the same vegetable type rather than across categories, indicating reliable class distinction but weaker freshness discrimination. Classification accuracy exceeded 85% for most categories, consistent with earlier detection results. These findings confirm the effectiveness of BYOL-learned features while underscoring the need to improve sensitivity to subtle freshness differences and reduce ambiguity among visually similar damaged states.

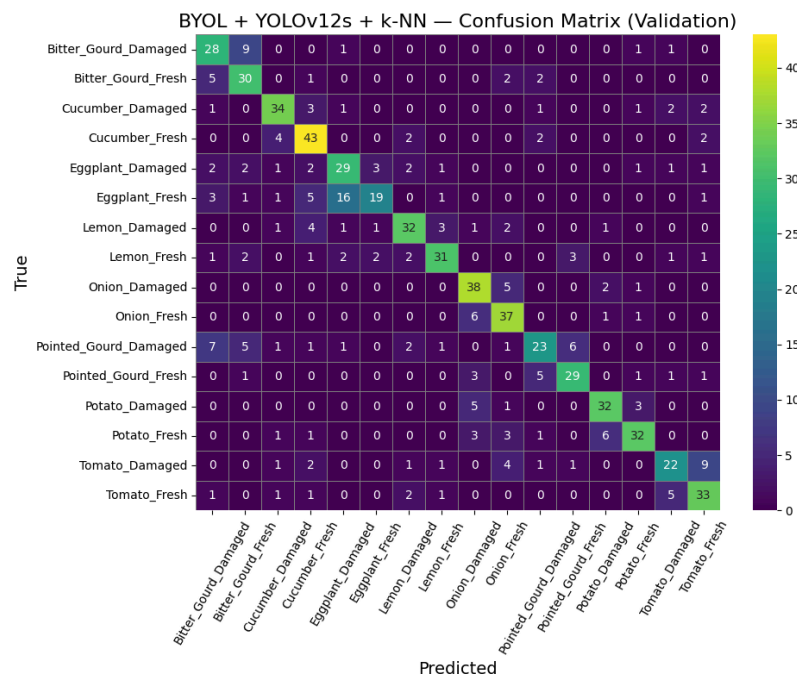
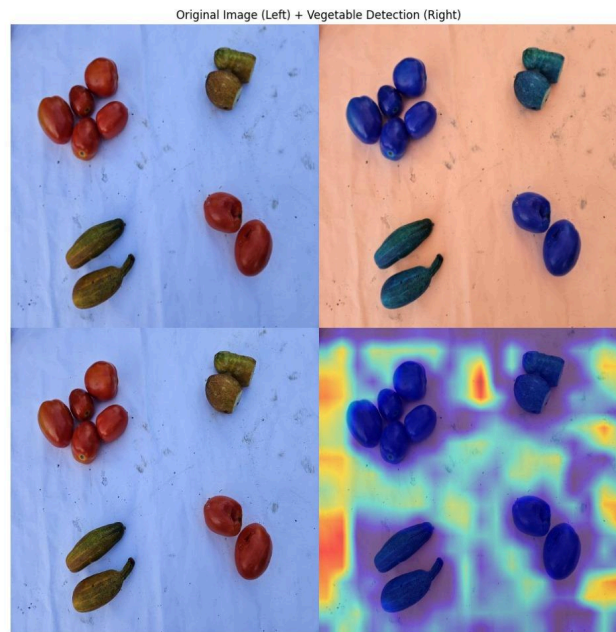


Figure 16: KNN Confusion Matrix after BYOL Feature Extraction

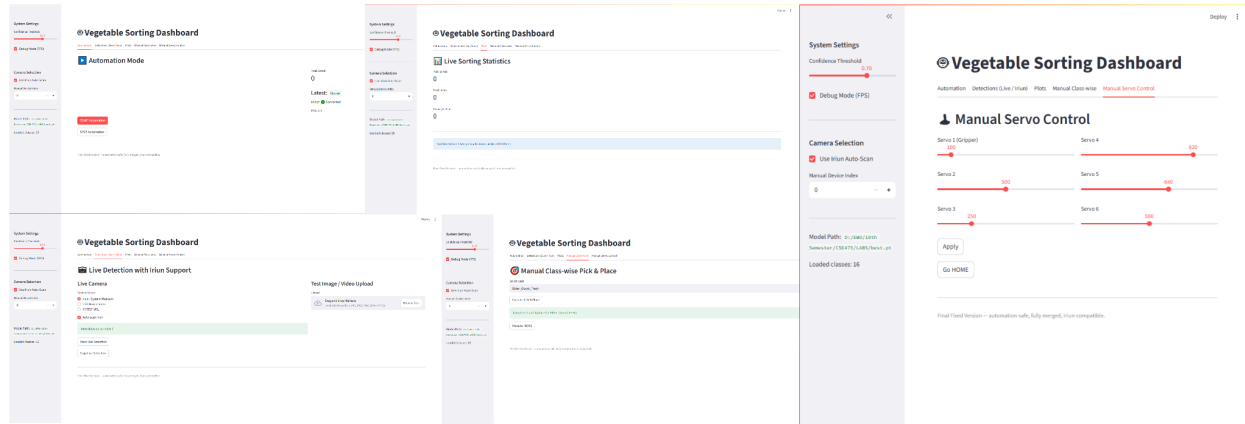
## 7. Explainable AI Techniques

Explainable AI (XAI) methods were used to interpret how the classification model distinguishes between fresh and damaged vegetables. These techniques help verify whether the model attends to relevant visual cues such as surface texture, color variation, and structural defects. Figure 17 shows an EigenCAM visualization applied to a sample input. The left panel displays the original image, while the right panel presents the corresponding heatmap. The model's attention is concentrated on blemished regions of the vegetable, avoiding the background and edges. This localized activation confirms that the model associates visible spoilage with the “damaged” label. Heatmap intensity reflects feature importance: brighter areas indicate stronger influence on the final prediction. In most cases, the model focuses on semantically meaningful zones, aligning with human judgment. However, occasional activation in background regions suggests sensitivity to noise, pointing to potential areas for refinement in preprocessing or augmentation. Overall, XAI visualizations enhance trust in the model's behavior and support its deployment in real-world sorting tasks. By revealing the basis of each decision, these tools ensure that predictions are both accurate and interpretable.



**Figure 17: EigenCAM Visualization and HeatMap**

## 8. Real-Time Web Application Dashboard



**Figure 18: Streamlit Dashboard**

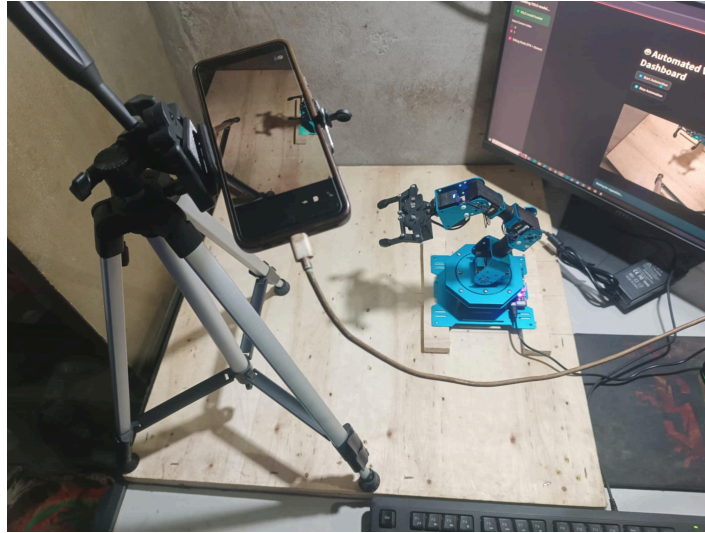
Figure 18 shows the integrated web-based dashboard that was implemented for the operation and monitoring of the entire vegetable-sorting system. The interface integrates various functional modules together, which can support seamless interactions among vision-based detection, robotic actuation, and system-level configuration. The Automation Panel supplies one-click control for initiating and terminating end-to-end automated sorting, processing in real time from camera stream inputs down to object handling operations. The Live Detection Module allows both webcam and smartphone-based video feeds, with further support for uploading image and video files; it allows users to preview the results, download annotated outputs, and configure detection parameters such as thresholds in confidence and debugging overlays.

The dashboard also includes a dedicated Plot Visualization Module that allows users to inspect statistics on detection, class distributions, and historical system performance directly from the interface. For fine-grained object-wise execution, the Manual Class-wise Pick & Place Module reads class definitions from the system's `data.yaml` file to enable focused manipulation of specific object categories. Complementing the automated pipeline, the Manual Servo Control Panel supports tuning through sliders for all six servo channels including the gripper which allows detailed adjustment in actuation and safe calibration for the robotic arm. Altogether, this dashboard represents a centralized control environment that unifies perception, decision-making, visualization, and actuation into one coherent and operator-friendly system.

## 9. Robotic arm Performance

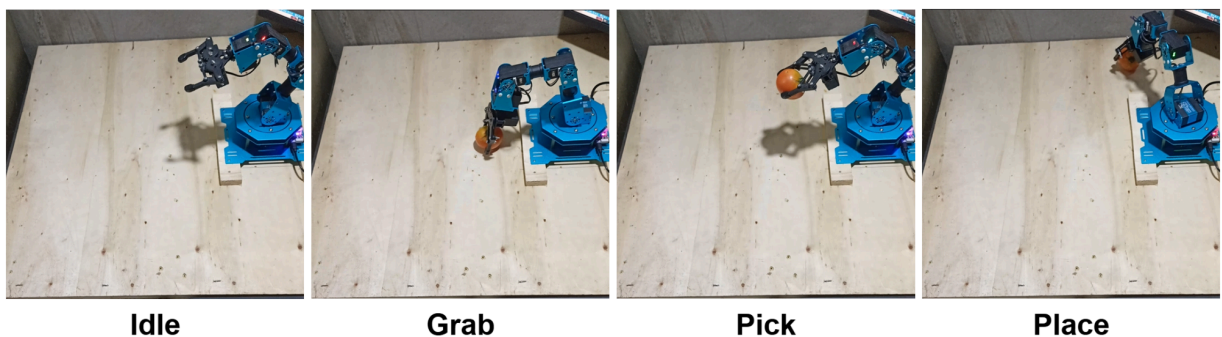
Figure 19 shows the experimental platform designed for real-time object detection and robotic manipulation. A smartphone mounted on a tripod serves as the primary vision sensor to capture top-down video streams of the workspace. The visual data is sent to a host computer, where a custom YOLO-based SSL inference pipeline processes the incoming frames for object localization and class prediction. The

robotic arm is anchored at the center of the workstation onto a wooden base to perform the pick-and-place action based on the coordinates of the detected objects. In addition, the computer monitor will display live inference results such as bounding boxes and system logs, providing immediate verification of the perception-to-action loop. This integrated setup demonstrates a complete closed-loop system that combines mobile vision, edge-based computation, and robotic actuation for real-time autonomous manipulation tasks.



**Figure 19: Experimental Platform**

Figure 2 depicts the sequential operational workflow performed by the vision-guided robotic arm in executing the object manipulation task. It starts with an Idle state where the arm is in a stationary position, awaiting the detecting trigger from the vision module. By identifying the target object, the arm reaches the Grab phase, by which the end-effector aligns with the location of the object and initiates the grasping action. The Pick phase elevates the object from the surface of the workspace while the arm stabilizes the orientation to minimize slippage. Finally, in the Place phase, the arm moves the object to the specific region for dropping off the object and releasing it. This represents a complete perception-to-action pipeline, where the system takes autonomous detection, approach, grasp, lift, and relocation of objects within an environment.



**Figure 20: Sequential Operational Workflow**

# Conclusion

This work presented the development of a vision-guided robotic system for automated vegetable sorting that effectively integrated self-supervised learning with low-cost robotic manipulation. By using the DINOv2-style and BYOL-based architectures, it achieved strong detection performance and showed reliable differentiation between fresh and damaged produce across various conditions of imaging. Confusion matrix analyses demonstrated that class recognition was highly robust, and explainable AI visualizations verified that models indeed attend to semantically relevant regions, therefore supporting interpretability and trust. The effective integration of the robotic arm enabled the successful performance of pick-and-place operations with high consistency, hence validating the system's capability for real-time and autonomous manipulation tasks. Overall, the findings have shown the practicality and efficiency of SSL-based approaches in reducing dependency on large labeled datasets while sustaining high accuracy. This provides a scalable and sustainable pathway toward automated food processing.

Despite these promising results, some limitations still remain that are worth further investigation. Freshness discrimination among visually similar vegetables, such as bitter melon and pointed melon, was challenging and required more refined SSL architectures, targeted ablation studies, and improved hyperparameter tuning. More importantly, this would require improving the diversity of the dataset using advanced augmentation techniques, including synthetic lighting variations, occlusion simulation, and texture-preserving transformations, since robust generalization in real-world agricultural environments depends on that. From a robotics perspective, further optimization of the end-effector design along with motion-planning algorithms will help reduce damage and facilitate irregular object geometries. Furthermore, real-time monitoring, interactive explainable AI, and modular control interfaces need to be integrated into an operational dashboard in order to extend system usability and deployment readiness. All of these will go a long way in further strengthening the robustness of the system and accelerating its transition toward large-scale commercial adoption in automated sorting pipelines.

# Reference

1. Gul, N., Muzaffar, K., Shah, S. Z. A., Assad, A., Makroo, H. A., & Dar, B. N. (2024). Deep learning hyperspectral imaging: a rapid and reliable alternative to conventional techniques in the testing of food quality and safety. *Quality Assurance and Safety of Crops & Foods*, 16(1), 78-97.
2. A. R. Kulkarni, A. Gupta, A. K. Yadav, A. Kumar, Fruits/vegetable classification using deep learning and iot, *International Research Journal of Modernization in Engineering Technology and Science* 04 (05) (2022) 719–726.
3. W. J. Feng, Y. L. Zhao, Y. Xie, L. Wei, Enhanced yolov5s-based apple detection for harvesting robots using a multi-rotated box algorithm, *Informatica* 49 (18) (2025) 191–204. doi:10.31449/inf.v49i18.7918.
4. Y. Liu, C. Wei, S.-C. Yoon, X. Ni, W. Wang, Y. Liu, D. Wang, X. Wang, X. Guo, Development of multimodal fusion technology for tomato maturity assessment, *Sensors* 24 (8) (2024) 2467. doi:10.3390/s24082467.
5. R. Zhao, Y. Zhu, Y. Li, Cla: A self-supervised contrastive learning method for leaf disease identification with do-main adaptation, *Computers and Electronics in Agriculture* (2023). doi:10.1016/j.compag.2023.107967.
6. N. K. Uppalapati, B. T. Walt, A. J. Havens, A. Mahdian, G. Chowdhary, G. Krishnan, A berry picking robot with a hybrid soft-rigid arm: Design and task space control, *Robotics: Science and Systems* (2020).
7. S. A. Bacha, A. Taheri-Garavand, R. Ben Ayed, Tomato maturity recognition with convolutional transformers, *Scientific Reports* 13 (2023) 22885. doi:10.1038/s41598-023-50129-w.
8. L. Zhang, J. Liu, Y. Wei, D. An, X. Ning, Self-supervised learning-based multi-source spectral fusion for fruit quality evaluation: A case study in mango fruit ripeness prediction, *Information Fusion* (2024). doi:10.1016/j.inffus.2024.102814.
9. C. Wang, X. Yao, F. Ding, Z. Yu, A trajectory planning method for a casting sorting robotic arm based on a nature-inspired genghis khan shark optimized algorithm, *Mathematical Biosciences and Engineering* 21 (2) (2024) 3364–3390. doi:10.3934/mbe.2024149.
10. J. Kim, G. Kim, R. Yoshitoshi, K. Tokuda, Real-time object detection for edge computing-based agricultural automation: A case study comparing the yolox and yolov12 architectures and their performance in potato harvesting systems, *Sensors* 25 (15) (2025) 4586. doi:10.3390/s25154586.
11. F. Ali, W. Tariq, A. Razzaq, A. Rehman, S. Sarfraz, N. A. Rajput, S. Ali, K. Fatima, S. Jameel, N. Liaqat, Z. Akash, From sensors to insights: The fusion of ai, edge computing, and precision

agriculture, *Journal of Agriculture and Biology* 03 (1) (2025) 160–170. doi:10.55627/agribiol.003.01.1067.

12. Khanna, S., Chattopadhyay, C., & Kundu, S. (2024). Enhancing fruit and vegetable detection in unconstrained environment with a novel dataset. *Scientia Horticulturae*, 338, 113580.
13. Li, H., & Shi, F. (2024). A DETR-like detector-based semi-supervised object detection method for Brassica Chinensis growth monitoring. *Computers and Electronics in Agriculture*, 219, 108788.
14. Bu, X., Wu, Y., Lv, H., & Yu, Y. (2025). Fast and Accurate Detection of Forty Types of Fruits and Vegetables: Dataset and Method. *Agriculture*, 15(7), 760.
15. Casado-García, A., Heras, J., Milella, A., & Marani, R. (2022). Semi-supervised deep learning and low-cost cameras for the semantic segmentation of natural images in viticulture. *Precision Agriculture*, 23(6), 2001-2026.
16. Kirill Bogomasov and Stefan Conrad. 2024. Efficient Iterative Semi-supervised Object Detection Training for Counting of Fruits and Vegetables. In Proceedings of the 2023 7th International Conference on Advances in Artificial Intelligence (ICAAI '23). Association for Computing Machinery, New York, NY, USA, 133–137.
17. Barry Benator, "Chapter 6 Cost Estimating," in Project Management and Leadership Skills for Engineering and Construction Projects , River Publishers, 2003, pp.73-100.
18. R. Johanson, C. Wilms, O. Johannsen and S. Frintrop, "S3AD: Semi-supervised Small Apple Detection in Orchard Environments," 2024 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), Waikoloa, HI, USA, 2024, pp. 7061-7070, doi: 10.1109/WACV57701.2024.00692.
19. J. Zhou, J. Zhang, J. Chu, H. Huang and Y. Sun, "MemoryTeacher: Memory Fusion and CARF for Semi-supervised Pest Region Detection," 2024 7th International Conference on Advanced Algorithms and Control Engineering (ICAACE), Shanghai, China, 2024, pp. 1505-1508, doi: 10.1109/ICAACE61206.2024.10549614.
20. X. Xu, D. Zou, J. Zou, S. Xia and W. Sheng, "MulMatch-CL: A Semi-Supervised Teacher-Student Framework for Robust Crop Segmentation in UAV Imagery," in IEEE Access, vol. 13, pp. 122914-122927, 2025, doi: 10.1109/ACCESS.2025.3586498.
21. J. Yang, Z. Wang, D. Chen, S. Xiao and A. Taher Azar, "A Joint Network of Super-Resolution and Active Learning for Agricultural Land Classification," in IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, vol. 17, pp. 13669-13677, 2024, doi: 10.1109/JSTARS.2024.3432976.

22. A. Ter-Sarkisov, R. Ross and J. Kelleher, "Bootstrapping Labelled Dataset Construction for Cow Tracking and Behavior Analysis," 2017 14th Conference on Computer and Robot Vision (CRV), Edmonton, AB, Canada, 2017, pp. 277-284, doi: 10.1109/CRV.2017.25.
23. Ci, J., van Henten, E. J., Wang, X., Burusa, A. K., & Kootstra, G. (2025). SSL-NBV: A self-supervised-learning-based next-best-view algorithm for efficient 3D plant reconstruction by a robot. *Computers and Electronics in Agriculture*, 233, 110121.
24. Ci, J., van Henten, E. J., Wang, X., & Kootstra, G. (2025). Robotic self-supervised local view planning for efficient and scalable 3D plant reconstruction across varying plant sizes. *Computers and Electronics in Agriculture*, 238, 110839.
25. Choi, T., Would, O., Salazar-Gomez, A., Liu, X., & Cielniak, G. (2024). Channel randomisation: Self-supervised representation learning for reliable visual anomaly detection in speciality crops. *Computers and Electronics in Agriculture*, 226, 109416.
26. Liu, Y., Zhou, S., Wu, H., Han, W., Li, C., & Chen, H. (2022). Joint optimization of autoencoder and Self-Supervised Classifier: Anomaly detection of strawberries using hyperspectral imaging. *Computers and Electronics in Agriculture*, 198, 107007.
27. Steinbrener, J., Dimitrievska, V., Pittino, F., Starmans, F., Waldner, R., Holzbauer, J., & Arnold, T. (2023). Learning metric volume estimation of fruits and vegetables from short monocular video sequences. *Heliyon*, 9(4).
28. Saleh, A., Olsen, A., Wood, J., Philippa, B., & Azghadi, M. R. (2025). Semi-supervised weed detection for rapid deployment and enhanced efficiency. *Computers and Electronics in Agriculture*, 236, 110410.
29. Li, R., Wang, X., Cui, Y., Xu, Y., Zhou, Y., Tang, X., Jiang, C., Song, Y., Dong, H., & Yan, S. (2025). A Semi-Supervised Diffusion-Based Framework for Weed Detection in Precision Agricultural Scenarios Using a Generative Attention Mechanism. *Agriculture*, 15(4), 434. <https://doi.org/10.3390/agriculture15040434>
30. Chen, Z., Liu, F., Zhu, X., Qi, Y., & Ghavamzadeh, M. (2025). Preference Optimization via Contrastive Divergence: Your Reward Model is Secretly an NLL Estimator. ArXiv, abs/2502.04567. <https://doi.org/10.48550/arxiv.2502.04567>.(Chen et al., n.d., 20)
31. Hu, G., Zhao, Y., Bao, W., & Zhang, D. (2025). A semi-supervised detection method for multi-scale tea geometrid by integrating intra-and inter-class information in natural scene images. *Computers and Electronics in Agriculture*, 237, 110645.

32. Liu, J., Hu, Y., Su, Q., Guo, J., Chen, Z., & Liu, G. (2024). Semi-Supervised One-Stage Object Detection for Maize Leaf Disease. *Agriculture*, 14(7), 1140. <https://doi.org/10.3390/agriculture14071140>
33. Saleh, A., Hatano, S., & Azghadi, M. R. (2025). Weed Detection in Challenging Field Conditions: A Semi-Supervised Framework for Overcoming Shadow Bias and Data Scarcity. *arXiv preprint arXiv:2508.19511*.
34. H. Wang et al., "CC-SSL: A Self-Supervised Learning Framework for Crop Classification With Few Labeled Samples," in *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 15, pp. 8704-8718, 2022, doi: 10.1109/JSTARS.2022.3211994.
35. Najafian, K., Ghanbari, A., Stavness, I., Jin, L., Shirdel, G., & Maleki, F. (2021). A Semi-self-supervised Learning Approach for Wheat Head Detection using Extremely Small Number of Labeled Samples. 2021 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW), 1342-1351. <https://doi.org/10.1109/iccvw54120.2021.00155>.
36. A. A. Mamun, D. Ahmedt-Aristizabal, M. Zhang, M. Ismail Hossen, Z. Hayder and M. Awrangjeb, "Plant Disease Detection Using Self-Supervised Learning: A Systematic Review," in *IEEE Access*, vol. 12, pp. 171926-171943, 2024, doi: 10.1109/ACCESS.2024.3475819.
37. Gupta, M. R., & Sobti, A. (2025). Time2Agri: Temporal Pretext Tasks for Agricultural Monitoring. *arXiv preprint arXiv:2507.04366*.
38. Wang, W., Zhu, A., Wei, H., & Yu, L. (2024). A novel method for vegetable and fruit classification based on using diffusion maps and machine learning. *Current Research in Food Science*, 8. <https://doi.org/10.1016/j.crfs.2024.100737>.
39. Chai, A. Y. H., Lee, S. H., Tay, F. S., Bonnet, P., & Joly, A. (2024). Beyond supervision: Harnessing self-supervised learning in unseen plant disease recognition. *Neurocomputing*, 610, 128608.
40. Khoiruddin, M., & Tena, S. (2024). Fruit and Vegetable Classification using Convolutional Neural Network with MobileNetV2. *Journal of Applied Research In Computer Science and Information Systems*. <https://doi.org/10.61098/jarcis.v2i2.197>.